Information Fusion 134 (2026) 104331



Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus



Full Length Article



Rebalancing with calibrated sub-classes (RCS): A statistical fusion-based framework for robust imbalanced classification across modalities

Priyobrata Mondal, Faizanuddin Ansari, Swagatam Das*

Electronics and Communication Sciences Unit, Indian Statistical Institute, Kolkata, 700108, West Bengal, India

ARTICLE INFO

Keywords:
Class imbalance
Distribution calibration
Synthetic sample generation

ABSTRACT

Class imbalance, where certain classes have insufficient data, poses a critical challenge for robust classification, often biasing models toward majority classes. Distribution calibration offers a promising avenue to address this by estimating more accurate class distributions. In this work, Rebalancing with Calibrated Sub-classes (RCS) - a novel distribution calibration framework for robust imbalanced classification has been proposed. RCS aims to fuse statistical information from the majority and intermediate class distributions via a weighted mixture of Gaussian components to estimate minority class parameters more accurately. An encoder-decoder network is trained to preserve structural relationships in imbalanced datasets and prevent feature disentanglement. Post-training, encoder-extracted feature vectors are leveraged to generate synthetic samples guided by the calibrated distributions. This fusion-based calibration effectively mitigates overgeneralization by incorporating neighborhood distribution information rather than relying solely on majority-class statistics. Extensive experiments on diverse image, text, and tabular datasets demonstrate that RCS consistently outperforms several baseline and state-of-the-art methods, highlighting its effectiveness and broad applicability in addressing real-world imbalanced classification challenges. RCS code is available at https://github.com/priyomondal/RCS.

1. Introduction

Deep learning has achieved remarkable success across a wide spectrum of domains, including computer vision, natural language processing, and healthcare [1]. Its ability to learn hierarchical and highly expressive representations from large-scale data has fundamentally transformed modern data-driven modeling. However, the effectiveness of deep neural networks is strongly conditioned on the assumption that training data are sufficiently abundant and relatively balanced across classes. In practice, this assumption is frequently violated. Real-world data collection processes are often influenced by the rarity of events, acquisition cost, privacy constraints, or naturally skewed distributions, leading to significant class imbalance.

Despite their strong representational power, deep learning models often struggle when trained on imbalanced datasets, where certain classes are severely underrepresented. This imbalance results in biased optimization dynamics, where the empirical risk minimization framework disproportionately favors majority classes, leading to skewed decision boundaries and degraded minority-class performance [2]. The issue is particularly critical in real-world applications such as medical diagnosis and fraud detection, where minority classes frequently correspond to rare yet high-risk events. In such high-stakes scenarios, misclassification of minority samples may lead to severe practical consequences, underscoring the need for principled imbalance-handling strategies that go beyond heuristic reweighting.

Oversampling techniques have emerged as one of the most widely adopted solutions to mitigate class imbalance. Classical approaches, such as the Synthetic Minority Oversampling Technique (SMOTE) [3], generate synthetic minority samples by interpolating between nearest neighbors in the feature space. While effective for low-dimensional tabular data, these methods encounter substantial limitations in high-dimensional settings such as image data, where the curse of dimensionality weakens distance-based interpolation reliability.

Recent extensions aim to enlarge the feasible space of synthetic sample generation beyond strict linear interpolation. For example, ISMOTE [4] expands synthetic generation by adaptively perturbing interpolated samples according to local spatial relationships. By generating instances not only between minority samples but also within surrounding neighborhoods, ISMOTE alleviates excessive density concentration. Nevertheless, such approaches remain fundamentally dependent on Euclidean geometry in the original feature space and do not explicitly model structured class-conditional distributions, which limits their robustness in semantically complex or highly non-linear representation spaces.

* Corresponding author.
E-mail address: swagatamdas19@yahoo.co.in (S. Das).

https://doi.org/10.1016/j.inffus.2026.104331
Received 3 October 2025; Received in revised form 21 March 2026; Accepted 25 March 2026
Available online 4 April 2026
1566-2535/© 2026 Published by Elsevier B.V.

To address high-dimensional limitations, deep generative models such as Generative Adversarial Networks (GANs) [5] and Variational Auto-Encoders (VAEs) [6] have been incorporated into imbalance learning frameworks, including GAMO [7], DGC [8], DeepSMOTE [9], and DGCMM [10]. These models synthesize minority samples in learned latent spaces and have demonstrated improvements over purely interpolation-based strategies. However, they introduce new challenges, including training instability, mode collapse, limited diversity under severe scarcity, and substantial computational overhead. Moreover, generative adversarial optimization does not inherently guarantee distributional calibration with respect to majority classes, which may result in visually plausible yet statistically inconsistent samples.

Li et al. introduce a complemented subspace regularization and adaptive class-weighted collaborative representation [11] framework to improve minority-class discrimination while maintaining closed-form computational efficiency. The method improves classifier-level discrimination but does not explicitly model or calibrate class-conditional distributions, limiting robustness when minority classes exhibit multimodal [12] or structurally complex distributions that require distribution-aware synthesis. Groupwise label enhancement strategy [13] within the BLS framework has been proposed to jointly preserve intraclass similarity and interclass disparity through flexible regression targets and groupwise constraints. Although label flexibility improves discriminability, the approach focuses on label-space optimization rather than distribution-level modeling, leaving minority data scarcity and distribution mismatch unaddressed in latent feature space. Jin et al. put forth an imbalanced BLS framework [14] combining genetic-programming-based feature optimization, latent label relaxation, and adaptive sample weighting to reduce majority bias. Despite adaptive weighting and relaxed supervision, the method lacks a principled mechanism for estimating minority distributions from majority and intermediate statistics, which can limit authenticity and diversity in synthesized samples under severe imbalance.

Balanced representation across classes is crucial for reliable classifier generalization. Effective oversampling must satisfy two fundamental principles:

Authenticity: Generated samples must faithfully belong to the minority distribution without introducing artificial artifacts that distort class semantics.

Pattern Coverage: Synthesized samples must capture the intrinsic diversity within the minority class to avoid over-concentration and limited generalization.

Existing interpolation-based and generative approaches often satisfy one principle at the expense of the other. Linear interpolation may preserve local authenticity but lacks distributional diversity, whereas adversarial generation may improve diversity without guaranteeing calibrated statistical consistency. This gap motivates the need for a representation-consistent and statistically grounded oversampling framework.

To address these limitations, we propose the Rebalancing with Calibrated Sub-classes (RCS) framework, a two-stage methodology that integrates disentangled representation learning with structured distribution calibration. The central idea is to construct a latent space where class representations are disentangled and well separated, and then perform subclass-level Gaussian calibration using majority and intermediate class statistics to estimate minority distributions more faithfully.

Our approach first constructs disentangled latent representations from imbalanced data. The objective is to extract feature vectors such that separability in data space translates into separability in latent space. Below, we formally define disentangled representation.

Definition 1. Let X be the set of data points, $\mathscr{L}$ be the space of latent representations, and $\mathscr{E} : X \mapsto \mathscr{L}$ be a function mapping data points to feature space. Let $U_1, U_2, \cdots, U_K \subset X$ represent elements from different classes. Without loss of generality, suppose $U_i, U_j \subset X$ denote datapoints from the ith and j^{th} classes respectively, and $\mathscr{E}(U_i) = \tilde{U}_i$, $\mathscr{E}(U_j) = \tilde{U}_j$. Let $q_1, q_2, \cdots, q_K$ be the class conditional distributions for the latent representations $\tilde{U}_1, \tilde{U}_2, \cdots, \tilde{U}_K$. Then $\{z \sim q_i : z = \mathscr{E}(x) \text{ and } x \in U_i\} \cap \{z \sim q_j : z = \mathscr{E}(x) \text{ and } x \in U_j\} = \phi$.

Such disentangled representations ensure non-overlapping latent distributions across classes, directly supporting the authenticity principle.

The proposed RCS framework consists of two stages:

Stage 1: Disentanglement: We train a regularized autoencoder combining reconstruction loss, classifier supervision, and supervised contrastive learning to enforce discriminative and well-separated latent representations. This regularization ensures intra-class compactness and inter-class separation, yielding robust subclass structures (Fig. 1b, 1d).

Stage 2: Synthesis: Assuming approximate Gaussianity within latent subclasses, we perform structured distribution calibration by leveraging majority and intermediate class statistics to estimate minority class distributions. This calibrated synthesis generates latent samples that preserve authenticity while expanding pattern coverage (Fig. 1e).

By combining representation disentanglement with subclass-level statistical calibration, RCS bridges deep feature learning and principled probabilistic modeling. This integration enables reliable minority distribution estimation without adversarial instability, while remaining computationally efficient compared to generative alternatives.

The major contributions of our research are as follows:

(1) We introduce **a novel calibration method to estimate the distribution** of minority class samples (Section 3.3.2).

(2) Unlike traditional approaches focusing mainly on the majority class, the proposed RCS framework also incorporates intermediate classes (Section 3.3.1).

(3) A regularization strategy that enlarges inter-class margins and supports discriminative latent structuring.

(4) Extensive validation across image, text, and tabular datasets demonstrates superior performance and generalization capability.

The remainder of this paper is organized as follows: Section 2 reviews existing class imbalance techniques; Section 3 introduces our two-stage oversampling method; Section 4 reports results across datasets; Section 5 presents an ablation study; and Section 6 concludes with findings and future directions.

2. Related work

Approaches to class imbalance are broadly categorized into data-level, algorithm-level, and hybrid methods [15]. Data-level techniques, particularly oversampling, are a standard remedy. Classical oversampling algorithms, such as RWO [16], SMOTE [3] and its variants (e.g., Borderline-SMOTE [17], ADASYN [18]), generate synthetic minority samples via interpolation. While effective for low-dimensional data, their reliance on distance metrics makes them less suitable for high-dimensional domains like images. For complex data, deep generative models offer a powerful alternative. Models such as Variational Autoencoders (VAEs) [6] and Generative Adversarial Networks (GANs) [5] can synthesize high-fidelity minority samples. Conditional GANs [19] and BAGAN [20] generate class-specific samples, while integrated frameworks like GAMO [7] jointly optimize data generation and classifier training in an adversarial game. A significant advancement is DeepSMOTE [9], which performs oversampling in a trained autoencoder's latent space, bridging the gap between feature representation and sample generation. Other notable works include DGC [8] and DGCMM [10], which use deep generative processes with Bayesian inference to model complex class distributions.

Our work builds upon these deep generative oversampling strategies. However, unlike methods that generate samples using convex combinations of all minority instances (e.g., [7,21]), or those that do not fully account for the underlying data manifold, our approach proposes a more efficient and distribution-aware generation process. A **detailed discussion of related works** is provided in the Supplementary section S1.

What is Distribution Calibration? Distribution Calibration, in the context of few-shot learning introduced by Yang et al. in [22], refers

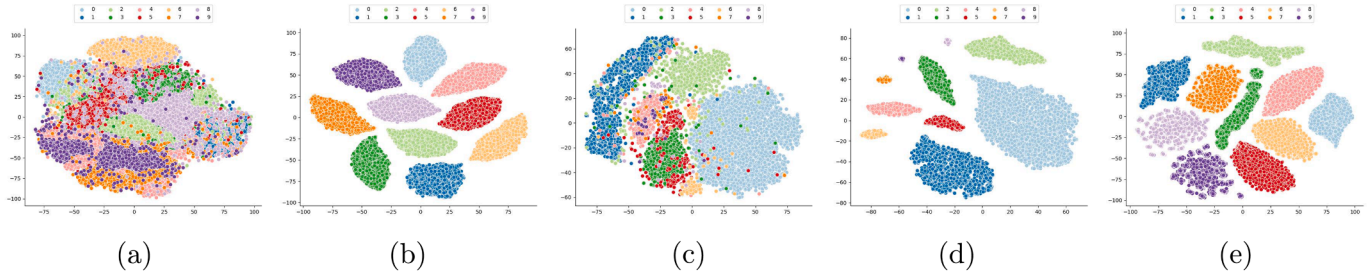


Fig. 1. Figures depict the t-SNE plots of feature vectors in the latent space for the MNIST dataset (a) Balanced Dataset with DeepSMOTE Autoencoder, (b) Balanced Dataset with our Autoencoder, (c) DeepSMOTE Autoencoder on Imbalanced Dataset, (d) Our Autoencoder on Imbalanced Dataset, and (e) RCS Oversampling on Imbalanced Dataset.

to a strategy for calibrating the distribution of classes with few training examples by transferring statistics from classes with sufficient examples. These calibrated statistics are then used to generate feature vectors corresponding to each element in the minority class by sampling from the calibrated Gaussian distributions. This allows for generating more diverse and accurate feature distributions, which can improve the performance of few-shot learning models. Vigneswaran et al. [23] proposed TailCalibX, which follows a similar strategy of estimating the parameters of the Gaussian Distribution corresponding to each sample in the minority class based on the nearest neighbor approach and samples an equal number of samples from a particular class for Imbalanced Datasets. Another calibration approach, as proposed in [24], to tackle imbalance, considers the concept of the fractional norm to calculate the k-nearest neighbor and adaptive weighing to calculate a calibrated distribution for the minority class. The existing calibration method's shortcomings in handling class imbalance lie in the fact that the method uses k-nearest classes for generating the samples in the minority class. By considering the nearest class feature vectors, the minority class elements are sampled so that they learn the nearest class in a better way than the furthest class, which in turn may affect the inherent characteristics of the underlying minority class. Hence, to tackle these shortcomings, this work presents a method that does not focus on the overall characteristics of the nearby class but considers the local characteristics of those classes.

3. Proposed methodology

3.1. Preliminaries

In Table 1 we described the notations we have used in this manuscript.

3.1.1. Autoencoder for feature representation

The autoencoder network consists of an encoder network ($\mathscr{E}$) and a decoder network ($\mathscr{D}$). Images from the image space (X) are fed into the encoder network ($\mathscr{E}$) to obtain latent representations in the latent space ($\mathscr{L}$). The decoder network ($\mathscr{D}$) takes these latent representations from the latent space ($\mathscr{L}$) as input to reconstruct the images in the reconstructed image space ($\hat{X}$). The objective is to make the original images X as similar as possible to the reconstructed images $\hat{X}$, such that $X \approx \hat{X}$.

3.1.2. Gaussian mixture model (GMM) for a class

We can cluster the data points belonging to a particular class in a dataset as a mixture of Gaussian distributions for a parametric data distribution. Let the latent vectors corresponding to the cth class be $L_c = \{l_{c1}, l_{c2}, \cdots, l_{cN_c}\}$. We perform GMM for class c on the latent feature vectors L_c with the number of components as ξ_c. Then the probability density for the sample l_{cj} can be represented as:

$$f(l_{cj}, \Theta) = \sum_{i=1}^{\xi_c} \alpha_i f_i(l_{cj}, \theta_i), \tag{1}$$

where α_i represents the mixture coefficients, the parameter space $\Theta = (\theta_1, \theta_2, \cdots, \theta_\xi)$, the Gaussian density function corresponding to the parameter θ_i is f_i.

Suppose we define the parameters corresponding to each component as $\theta_i = (\mu_i, \Sigma_i)$. Then GMM cluster, $f_i(l_{cj}, \theta_i)$ is represented by:

$$f_i(l_{cj}, \theta_i) = \frac{1}{\sqrt{2\pi|\Sigma_i|}} e^{-\frac{1}{2}(l_{cj}-\mu_i)^T \Sigma_i^{-1}(l_{cj}-\mu_i)}. \tag{2}$$

We define the number of elements in each component as S_i, where S_i is the ith element in the array S (the set containing the number of elements in each mixture component). For our experiments, we considered the number of elements in each component, S_i, to be greater than the number of elements in the class having the fewest elements (i.e., N_K). Hence, for the cth class the number of components $\xi_c = \frac{N_c}{N_K}$. When $\frac{N_c}{N_k}$ is non-integer, the number of Gaussian mixture components is determined as

$$\xi_c = \max\left(1, \text{round}\left(\frac{N_c}{N_k}\right)\right),$$

thereby ensuring that each class is represented by at least one component while maintaining proportionality with respect to class size.

3.2. Problem definition

Consider the training dataset $X = (x_i, y_i)_{i=1}^n$ for a K-multi-class imbalance classification problem, where n is the number of samples, x_i is the ith image data, and y_i is the corresponding label. $x_i \in \mathbb{R}^{C \times h \times w}$ and $y_i \in \mathcal{Y}$, where $\mathcal{Y} = \{1, 2, \cdots, c, \cdots, K\}$ set of labels and C, h, w are the number of channels, height, and the width of the images respectively. Let X_c denote the set of all data points in the class c, and the cardinality of the set X_c, denoted by $N_c = |X_c|$. The classes are labeled such that $N_1 \geq \cdots \geq \cdots \geq \cdots \geq N_K$. Given X, in our first stage, our objective is to learn a latent space $\mathscr{L} \in \mathbb{R}^d$ with a corresponding distribution $q(l)$. The set of latent vectors corresponding to the datasets X is represented by $\mathscr{L} = \{l_1, \cdots, l_i, \cdots, l_n\}$. The set of latent vectors corresponding to the particular class c is represented by L_c, and the corresponding mean and variance of latent vectors in class c are represented as μ_c and Σ_c. For the Image datasets, the Autoencoder network that we have utilized can be formally represented as $\mathscr{E}$ (encoder), $\mathscr{D}$ (decoder), and $\mathscr{H}$ (latent vector classifier). Using the learned latent representation, we aim to oversample the imbalanced feature vectors in the latent space using our proposed synthetic data oversampling algorithm. The reconstructed vectors obtained from the decoder are represented as $\hat{X}$, and the reconstructed vectors corresponding to the class c are represented by $\hat{X}_c$. We employ a discriminative classifier network, denoted as $\mathscr{F}$, for comparison with the state-of-the-art methods.

For the tabular data, we use the input data as a feature vector instead of obtaining the latent vectors from the trained autoencoder. Thus, the tabular data $X = (x_i, y_i)_{i=1}^n$ is same as $(l_i, y_i)_{i=1}^n$, where l_i is the latent vector or the feature vector. This disentangled latent structure forms the foundation for the distribution calibration mechanism introduced in the following section.

Table 1
Table of Notations.

Notations	Descriptions
X	Set of all Images, Tabular or Textual data
$\mathcal{Y}$	Set of all Labels
$\mathscr{L}$	Latent space feature vectors
x	An image data
y	Label corresponding to the image data x
$\mathbb{R}$	Set of real numbers
$\mathscr{M}$	Majority Class
$\hat{\mathcal{F}}$	Set of Intermediate Classes
$\mathcal{F}$	Intermediate Class $\mathcal{F} \in \hat{\mathcal{F}}$
$\hat{\mathfrak{m}}$	Set of Minority Classes
$\mathfrak{m}$	Minority Class $\mathfrak{m} \in \hat{\mathfrak{m}}$
d	dimension of the feature space
n	Number of samples in the training set
$\|.\|$	The cardinality of a set
$\mathcal{L}_{AC}$	Class Aware Loss
$\mathcal{L}_{CG}$	Classifier Guidance Loss
$\mathcal{L}_{CS}$	Supervised Contrastive loss
(μ'_i, Σ'_i)	Mean and variance of the majority class corresponding to the ith GMM component
$(\bar{\mu}_i, \bar{\Sigma}_i)$	Mean and variance of the intermediate class ($\mathcal{F}$) corresponding to the ith GMM component
$(\tilde{\mu}_i, \tilde{\Sigma}_i)$	Estimated Mean and variance corresponding to the ith element in the minority class
$\mu_{\mathscr{M}}$	Collection of the mean of each GMM component in the Majority class
$\Sigma_{\mathscr{M}}$	Collection of the variance of each GMM component in the Majority class
$\mu_{\hat{\mathcal{F}}}$	Collection of the mean of each GMM component in all the Intermediate classes
$\Sigma_{\hat{\mathcal{F}}}$	Collection of variance of each GMM component in all the Intermediate classes
$\mu_{\mathcal{F}}$	Collection of the mean of each GMM component in an Intermediate class $\mathcal{F}$, where $\mathcal{F} \in \hat{\mathcal{F}}$
$\Sigma_{\mathcal{F}}$	Collection of variance of each GMM component in an Intermediate class $\mathcal{F}$, where $\mathcal{F} \in \hat{\mathcal{F}}$
$\Sigma_{\mathfrak{m}}$	Covariance of the minority class $\mathfrak{m} \in \hat{\mathfrak{m}}$
$\Sigma_{\mathfrak{m}}^{\text{emp}}$	Empirical covariance of the minority class $\mathfrak{m} \in \hat{\mathfrak{m}}$
$(\hat{\mu}_i, \hat{\Sigma}_i)$	Mean and variance of the ith GMM component from the collection of the mean and variance of the Intermediate and Majority classes represented by $\hat{\mu}$ and $\hat{\Sigma}$ respectively.
K	Number of class
k	Number of neighbours
η	Imbalance tuner hyperparameter
ζ	the threshold cardinality which determines the classes that belong to the collection intermediate classes, $\mathcal{F}$, and the collections of minority classes $\mathfrak{m}$
N_i	Number of elements in the ith class
ξ_i	Number of components in the ith class
S_i	Number of elements in the ith component
p_i	Probability of a feature vector belonging to the ith class
$\hat{x}$	Representation of the reconstructed image x
l	a feature vector in the latent space $\mathscr{L}$
L_i	Feature vectors of the ith class from $\mathscr{L}$
$\tilde{L}$	Synthetic feature vectors sampled from a given normal distribution
$\hat{L}$	Set of oversampled feature vectors
$\hat{Y}$	Labels of the oversampled feature vectors
w_j	Weight associated to mean ($\hat{\mu}_j$) or variance ($\hat{\Sigma}_j$) for estimating the mean ($\tilde{\mu}_i$) and sigma ($\tilde{\Sigma}_i$) in the minority class
W	Collection of the weights w_i
n_i	The number of samples that have to be generated in the ith class
$\hat{n}_i$	The number of samples that have to be generated corresponding to each element in the ith class
t	Temperature hyperparameter for training the autoencoder
Q_i	Samples in the training set which have labels y_i
$Z(i)$	Samples in the training set excluding the ith sample
G	Group
G_i	i^{th} subgroup formed by decomposition of group G
$*$	Identity element in the group G
$\mathcal{T}$	Linear transformation $\mathcal{T} : X \to \mathscr{L}$
f, $\tilde{f}$	Equivariant map
$*_g$, $*_{sg}$	Group action
ρ	Map from the set of images to the set of labels

3.2.1. *Why disentanglement is necessary for distribution calibration*

The distribution calibration stage of RCS operates entirely in the latent feature space. Consequently, its validity depends critically on the statistical and geometric structure of that space. In particular, calibrated sampling using Gaussian mixture models implicitly assumes that latent representations are (i) class-separable and (ii) locally unimodal.

Disentangled representation learning in Stage 1 is therefore a necessary precondition for reliable calibration. Without disentanglement, latent features corresponding to different classes may overlap and exhibit multi-modal local neighborhoods, violating the assumptions required for meaningful estimation of class-conditional means and covariances. In such cases, calibration would combine incompatible semantic factors, leading to synthetic samples that blur decision boundaries and amplify majority-class bias.

By enforcing explicit class separation and equivariance-based constraints, the disentangled autoencoder learns a latent space in which samples from each class occupy distinct and compact regions. This ensures that the imbalance present in the original dataset is faithfully reflected in the latent space rather than obscured by representational entanglement. As a result, local neighborhoods around minority samples become statistically coherent and approximately Gaussian, enabling principled subclass-level calibration.

This structural property of the latent space is essential for the calibration step, which constructs new minority distributions by combin-

ing statistics from neighboring Gaussian components and the minority sample itself. The disentanglement stage thus guarantees that the calibrated mean and covariance estimates are both statistically meaningful and class-preserving, providing a foundation for the oversampling procedure in Stage 2.

The sequence of the subsequent sections is detailed as follows, subsection 3.3 outlines our data calibration method, and Subsection 3.4 expresses the details of training the Autoencoder.

3.3. RCS oversampling method

Our proposed oversampling algorithm will oversample/generate new latent vectors of the minority classes in the latent space. The algorithm tries to estimate the statistics related to the minority classes using the statistics of the majority class and the intermediate classes. This methodology will help meet our second principal requirement - pattern coverage. When we use the feature vectors (for tabular data) and the latent vectors (for image data), both represented by $\mathscr{L}$, we refer to them collectively as feature vectors for simplicity and to avoid confusion. In this section, the term feature vectors will be used to denote $\mathscr{L}$, and all definitions and operations applied to feature vectors will equally apply to the latent vectors used for image data. We assume that the feature vectors corresponding to different classes are well separated to facilitate effective oversampling.

In simpler terms, this approach takes information about the common patterns in the majority data and uses that knowledge to create new, valid data points for the minority class. This helps address the issue of having limited data for the minority class while ensuring that the generated data reflects the actual characteristics of that class. Here's an analogy: Imagine you have a collection of pictures of different colored dogs. There are many pictures of brown dogs (the majority class), but only a few pictures of black dogs (the minority class). This algorithm would analyse the feature vectors of the brown dogs (fur texture, body shape) and use that information to create new, realistic pictures of black dogs, filling the gaps in your data collection. Using statistics from classes with significantly more samples is beneficial because similar classes typically exhibit comparable means and variances in their feature representations [22]. As a result, the mean and variance of a Gaussian distribution can be applied to similar classes [25]. Furthermore, a sufficient number of samples improves the accuracy of these statistical estimates for any given class.

The proposed RCS framework also uses the statistics of classes with sufficient samples to estimate the statistics of minority classes. Instead of using statistics from the entire class, we utilise the statistics from the components or subclasses of the class with sufficient samples to better capture the local patterns. Our class division consists of three categories: the Majority class, the Intermediate class, and the Minority class. Intermediate classes typically comprise those classes with fewer samples than the majority class. The lower bound for the number of samples in the intermediate classes is determined by the hyperparameter ζ. This ensures that the classes included in the set of intermediate classes possess sufficient information to cover a diverse range of patterns within the class itself. We achieve this by leveraging the statistics of components or subclasses within those classes and using these statistics to generate new samples for these classes. The remaining classes are considered minority classes, and their distribution is calibrated by considering the statistics of components from nearby majority and intermediate classes. The subsequent sections discuss the description of class categorisation and the handling of different class details.

3.3.1. Class-level categorization for synthetic data generation

We start with the imbalanced feature vectors ($\mathscr{L}$) for the synthetic data generation. Based on the sample sizes of different classes in $\mathscr{L}$, we conglomerate the majority, intermediate, and minority classes. The term η refers to the imbalance tuner. The imbalance tuner hyperparameter is used to compute the threshold cardinality by defining ζ as $\frac{N_1}{\eta}$, where N_1 is the number of elements in the majority class. The threshold cardinality ζ decides whether a particular class would be considered an intermediate or minority class. The classes whose sample size is greater than ζ and less than N_1 (i.e., the cardinality of the majority class as defined in our problem definition) are coined as the Intermediate class, and the classes whose sample size is less than ζ are considered as the Minority class. Thus, based on the cardinality, classes are divided into three groups, namely, the majority class abbreviated as $\mathscr{M}$ and the Intermediate classes $\hat{\mathscr{I}}$ and the minority classes as $\hat{\mathfrak{m}}$. The three groups are handled in the following way:

(a) *Handling the Majority Class:* We perform the GMM clustering for the majority class ($\mathscr{M}$) to group the class into several mixture components and find the statistics mean (μ_i) and covariance (Σ_i) for them.

(b) *Handling the Intermediate Classes:* Consider an Intermediate class $\mathscr{I}$ from the set of all the intermediate classes $\hat{\mathscr{I}}$. For each intermediate class $\mathscr{I}$, we perform the GMM clustering on the feature space. We obtain the parameters mean (μ_i) and covariance (Σ_i) from each component in this feature space. Without loss of generality, let us assume that the corresponding intermediate class is the ith class. We first compute the number of samples that have to be generated in the ith class as $n_i = N_1 - N_i$. Corresponding to each component in the ith class, we generate $\hat{n}_i$ many samples, where $\hat{n}_i = n_i/\xi_i$, ξ_i being the total number of components in the ith class. In other words we generate $\hat{n}_i$ synthetic samples from the Gaussian distribution $\mathcal{N}(\mu_{ij}, \Sigma_{ij})$, where μ_{ij}, Σ_{ij} are the mean and covariance of the jth component in the ith class. Algorithm 1 provides a pseudocode of Generating Synthetic Samples in each Intermediate class $\mathscr{I}$.

Remark 1. The Algorithm 1 depicts the generation of synthetic samples. In this algorithm, the samples of the intermediate class are generated using each of the Gaussian Mixture components of the Intermediate Class. Gaussian mixture function returns the number of Gaussian components ξ_i, the number of samples in each component S_i, and $\tilde{\mu}_i = \{\tilde{\mu}_{i1}, \tilde{\mu}_{i2}, \cdots, \tilde{\mu}_{i\xi_i}\}$, $\tilde{\Sigma}_i = \{\tilde{\Sigma}_{i1}, \tilde{\Sigma}_{i2}, \cdots, \tilde{\Sigma}_{i\xi_i}\}$ represents the collection of the mean and variance corresponding to each component in the ith class, $\hat{\mu}$ stores all the mixture components associated to each Intermediate class ($\mathscr{I} \in \hat{\mathscr{I}}$), and the Majority class $\mathscr{M}$. $\hat{L}$ and $\hat{Y}$ represents the oversampled feature vectors and labels. This algorithm returns $\hat{L}$ and $\hat{Y}$ after the oversampling has been performed.

(c) *Handling the Minority Classes:* If the number of samples in a particular class is less than ζ, we consider those classes as minority classes. From the set of the minority classes $\hat{\mathfrak{m}}$ we consider $\mathfrak{m} \in \hat{\mathfrak{m}}$. Without loss of generality, let us assume that the corresponding minority class $\mathfrak{m}$ is the ith class. We first compute the number of samples that have to be generated in the ith class as $n_i = N_1 - N_i$. Corresponding to each datapoint in the ith class, we generate $\hat{n}_i$ many samples, where $\hat{n}_i = n_i/N_i$, N_i being the total number of samples in the ith class. For generating data in the minority classes, we use the data calibration technique discussed in the subsection 3.3.2.

Fig. 2 illustrates the handling of various classes, including the oversampling techniques applied to intermediate and minority classes.

3.3.2. Distribution calibration method for oversampling minority instances

Suppose l_i's are the latent vectors (or feature vectors) of the samples x_i's of the minority class $\mathfrak{m} \in \hat{\mathfrak{m}}$. Assume $(\mu'_1, \Sigma'_1), (\mu'_2, \Sigma'_2), \cdots, (\mu'_{\xi_s}, \Sigma'_{\xi_s})$

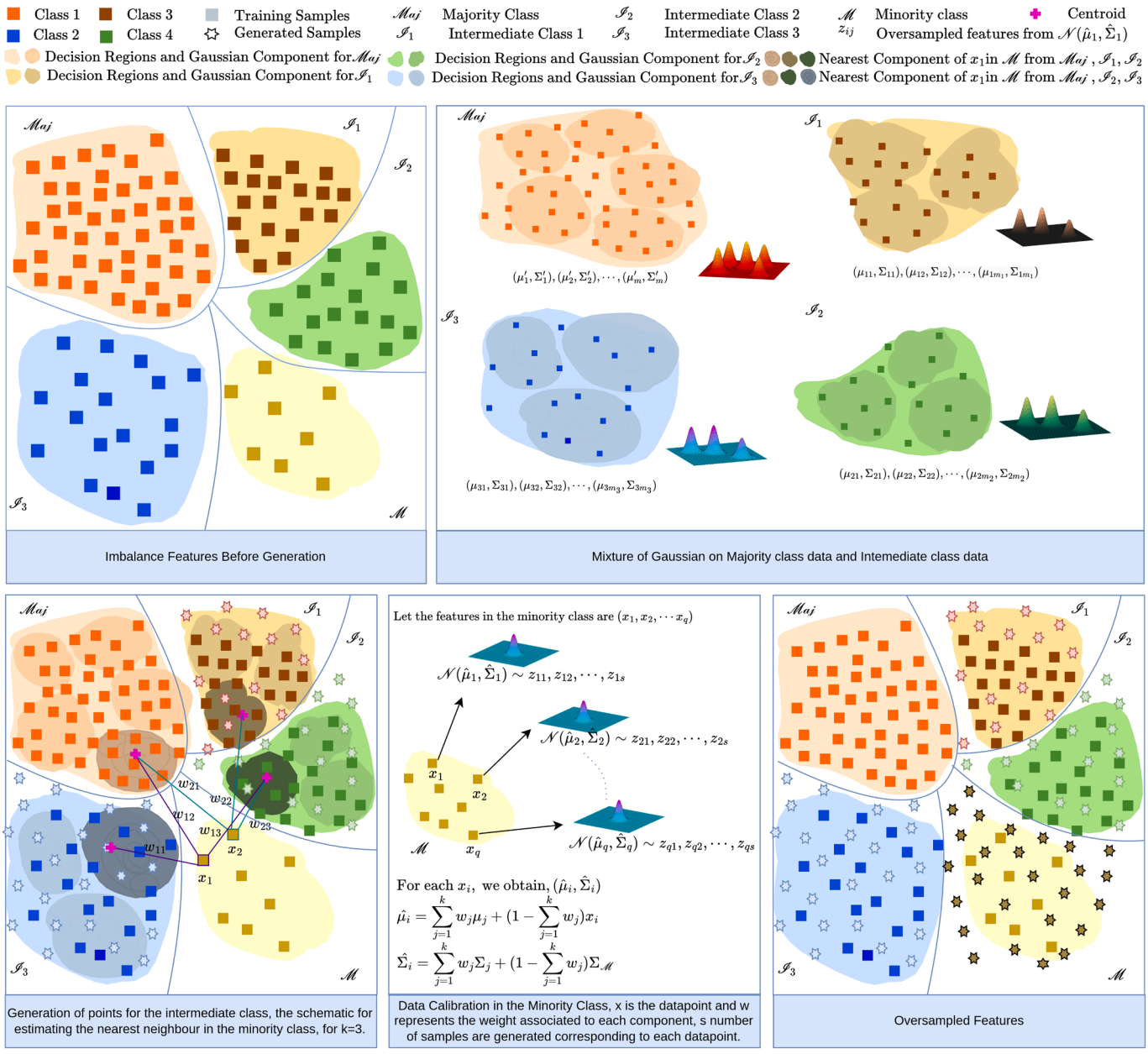


Fig. 2. Illustrations of our Data Calibration method, the first figure (from top left) represents the imbalanced feature vectors, the second figure represents the Gaussian Mixture components of the Majority class as well as the Intermediate classes, the third figure represents the oversampling in the latent space, and the visualization of the components participating in estimating the mean and variance corresponding to each point of the minority class from which synthetic feature vectors are sampled from, in the fourth figure corresponding to the estimated mean and variance we sample synthetic feature vectors from this Gaussian distribution and in the fifth figure we display the oversampled feature vectors.

be the statistics corresponding to the majority class $\mathscr{M}$, and $(\tilde{\mu}_{i1}, \tilde{\Sigma}_{i1}), (\tilde{\mu}_{i2}, \tilde{\Sigma}_{i2}), \cdots, (\tilde{\mu}_{\xi_i}, \tilde{\Sigma}_{\xi_i})$ are statistics corresponding to the intermediate classes in $\hat{\mathscr{F}}$. We conglomerate both the parameters of the majority class as well as intermediate classes to obtain $(\hat{\mu}, \hat{\Sigma}) = (\mu'_1, \Sigma'_1), (\mu'_2, \Sigma'_2), \cdots, (\mu'_s, \Sigma'_{\xi_s}) \bigcup (\tilde{\mu}_{i1}, \tilde{\Sigma}_{i1}), \ (\tilde{\mu}_{i2}, \tilde{\Sigma}_{i2}), \cdots, (\tilde{\mu}_{\xi_i}, \tilde{\Sigma}_{\xi_i}) = (\hat{\mu}_1, \hat{\Sigma}_1), (\hat{\mu}_2, \hat{\Sigma}_2), \cdots \cdots$. Among the means $\hat{\mu}$, we consider the k-neighbors corresponding to the datapoint l_i. Let $(\hat{\mu}_1, \hat{\Sigma}_1), (\hat{\mu}_2, \hat{\Sigma}_2), \ \cdots, (\hat{\mu}_k, \hat{\Sigma}_k)$ be statistics corresponding to the k neighbours to the data-point l_i among the means $\hat{\mu}_1, \hat{\mu}_2, \cdots, \hat{\mu}_k$. Then for each such l_i we calibrate the mean $(\bar{\mu}_i)$ as:

$$\bar{\mu}_i = \sum_{j=1}^{k} w_j \hat{\mu}_j + (1 - \sum_{j=1}^{k} w_j) l_i, \tag{3}$$

where, w_i's are the weights associated to each μ_i, and the weight associated to l_i is given by $(1 - \sum_{j=1}^{k} w_j)$.

The variance $(\bar{\Sigma}_i)$ associated with the minority class sample $(l_i \in \mathfrak{m})$ is given by

$$\bar{\Sigma}_i = \sum_{j=1}^{k} w_j \hat{\Sigma}_j + (1 - \sum_{j=1}^{k} w_j) \Sigma_{\mathfrak{m}}, \tag{4}$$

where, w_i's are the weights associated to each $\hat{\Sigma}_i$, and the weight associated to l_i is given by $(1 - \sum_{j=1}^{k} w_j)$.

All covariance matrices used during calibration are shrinkage-regularised, thereby ensuring positive semi-definiteness and numerical stability even in the presence of extreme minority-class sparsity.

Algorithm 1: Pseudocode of Generating Samples in Intermediate class ($\zeta \leq \#Samples < N_1$).

Input: $\mathscr{L}$ is set of feature vectors, Y set of labels corresponding to $\mathscr{L}$, N_i is the number of elements in the ith class and the class having the maximum number of elements is N_1, hyperparameter η.
Initialize: $\hat{L}, \hat{Y}, \hat{\mu}, \hat{\Sigma}, S$ as empty sets, ξ_i as the number of Gaussian Mixture components corresponding to the ith class with feature vectors L_i.
Output: The set of oversampled feature vectors $\hat{L}$ and the corresponding labels $\hat{Y}$ for the intermediate classes.

Compute:
$\zeta = \frac{N_1}{\eta}$
/* For loop running over all the classes */
for i **from** 1 **to** K **do**
  $L_i, Y_i \leftarrow$ obtains the feature vectors and labels of the ith class from $\mathscr{L}, Y$.
  $n_i = N_1 - N_i$
  if $N_i >= \zeta$ **and** $N_i < N_1$ **then**
    $\tilde{\mu}_i, \tilde{\Sigma}_i, \xi_i, S_i = GaussianMixture(L_i, Y_i)$/* Returns the Gaussian Mixture Components */
    $\hat{n}_i = n_i/\xi_i$
  /* For loop generating synthetic sampled from $\mathscr{N}(\tilde{\mu}_i, \tilde{\Sigma}_i)$ in the Intermediate Class $\mathscr{I}$ */
    for j **from** 1 **to** ξ_i **do**
      $\tilde{L}_j \leftarrow \hat{n}_i$ synthetic samples, sampled from the jth Gaussian component with mean $\tilde{\mu}_{ij}$ and sigma $\tilde{\Sigma}_{ij}$.
      $\tilde{Y}_j \leftarrow$ array of size $\hat{n}_i$ containing the ith class index.
      $\hat{L} \leftarrow \tilde{L}_j, \hat{Y} \leftarrow \tilde{Y}_j$
    end for
    $\hat{\mu} \leftarrow \tilde{\mu}_i, \hat{\Sigma} \leftarrow \tilde{\Sigma}_i$.
    $S \leftarrow S_i$
  end if
end for
return $\hat{L}, \hat{Y}$

We define the weight w_j corresponding to the jth Gaussian mixture component $(\hat{\mu}_j, \hat{\Sigma}_j)$ as:

$$w_j = \frac{1}{S_j}. \tag{5}$$

The weights w_j used in Eqs. (3) and (4) are defined in Eq. (5) as the inverse of the S_j, where S_j is the cardinality of the corresponding Gaussian Mixture component. Since $S_j > 0$, all the weights satisfy $w_j \geq 0$, and the residual coefficient $1 - \Sigma_j w_j$ is also non-negative. Consequently, the coefficients in Eqs. (3) and (4) form a valid convex combination consistent with Theorem 1, ensuring that the calibrated mean and covariance lie within the convex hull of the neighboring subclass statistics and the original minority sample.

Theorem 1 gives the theoretical consistency and shows that the estimated mean $\bar{\mu}_j$ is well-defined and states that the estimated mean corresponding to each feature in the minority class $\mathfrak{m}$ lies in the convex hull regions of the $\hat{\mu}_i's$ obtained from the Gaussian Mixture Models of the Majority Class $\mathscr{M}$ and each of the Intermediate Class $\mathscr{I} \in \hat{\mathscr{F}}$. The estimated mean $\bar{\mu}_j$ lies closer to l_j. This guarantees that the mean of the estimated distribution not only retains the inherent characteristics of the data but also acquires local characteristics of the nearby classes whose components lie closer to l_j.

Theorem 1. *Let $l_1, l_2, \cdots, l_n$ be the samples in the minority class $\mathfrak{m}$. Let $\bar{\mu}_j$ be the calibrating mean associated with each l_j, having the form $\bar{\mu}_j = l_j + \Sigma_{i=1}^{k}(\hat{\mu}_i - l_j)w_i$. Then $\bar{\mu}_j$ resides in the convex hull region of the $\hat{\mu}_i$'s and l_j. Moreover, the estimated mean $\bar{\mu}_j$ lies closer to l_j.*

Proof. The **proof of Theorem** 1 is provided in the Supplementary Section S2.1. □

Covariance Regularization and Robustness under Minority Sparsity: The covariance calibration step incorporates the minority-class covariance $\Sigma_{\mathfrak{m}}$, which may become unreliable when the minority class contains only a limited number of samples. To ensure numerical stability and statistical consistency, all covariance matrices used in the proposed framework are explicitly regularised prior to calibration.

Let $\Sigma_{\mathfrak{m}}^{\text{emp}}$ denote the empirical covariance estimated from the minority-class latent representations. A shrinkage formulation is adopted:

$$\Sigma_{\mathfrak{m}} = (1-\lambda)\Sigma_{\mathfrak{m}}^{\text{emp}} + \lambda I, \quad \lambda \in (0,1),$$

where I denotes the identity matrix.

For any non-zero vector v,

$$v^T\Sigma_{\mathfrak{m}}v = (1-\lambda)v^T\Sigma_{\mathfrak{m}}^{\text{emp}}v + \lambda||v||^2. \tag{6}$$

Since $\Sigma_{\mathfrak{m}}^{\text{emp}} \succeq 0$ and $\lambda > 0$, $v^T\Sigma_{\mathfrak{m}}v > 0$. Hence $\Sigma_{\mathfrak{m}}$ is strictly positive definite.

Let $\omega_{\min}(\Sigma_{\mathfrak{m}})$ denote the smallest eigenvalue of $\Sigma_{\mathfrak{m}}$, defined as

$$\omega_{\min}(\Sigma_{\mathfrak{m}}) = \min_{||v||=1} v^T\Sigma_{\mathfrak{m}}v.$$

Now,

$$\omega_{\min}(\Sigma_{\mathfrak{m}}) = (1-\lambda)\omega_{\min}(\Sigma_{\mathfrak{m}}^{\text{emp}}) + \lambda, \qquad \because \omega_{\min}(I) = 1,$$

which implies

$$\omega_{\min}(\Sigma_{\mathfrak{m}}) > 0.$$

Consequently,

$$||\Sigma_{\mathfrak{m}}^{-1}||_2 < \frac{1}{\lambda},$$

and the matrix inversion required for likelihood computation and sampling remains well defined. This formulation guarantees that $\Sigma_{\mathfrak{m}}$ remains strictly positive definite even in severely undersampled regimes, thereby preventing degeneracy and ensuring stable numerical operations.

The observed modality-dependent behavior of covariance shrinkage arises from fundamental differences in latent dimensionality, minority-class sample support, and the geometric structure of the learned feature representations across data modalities. In the case of textual datasets, the learned embeddings operate in relatively high-dimensional and sparsely populated latent spaces, where empirical covariance estimation is often sensitive to sampling noise and may become ill-conditioned under severe class imbalance. In such settings, shrinkage regularization

plays a crucial role by stabilizing covariance estimation, suppressing spurious correlations, and ensuring reliable sampling, which leads to improved empirical performance.

For image datasets, feature representations are learned using a convolutional autoencoder, which imposes strong spatial and structural inductive biases. As a result, the latent representations exhibit smoother and more compact geometric structure, and empirical covariance estimation remains sufficiently stable without requiring strong regularization. Similarly, most tabular datasets considered in this work lie in low- to moderate-dimensional regimes with adequate minority sample support, where empirical covariance estimation already captures meaningful variance structure. In these cases, shrinkage introduces an additional isotropic bias that may slightly oversmooth informative directions, leading to neutral or marginally reduced performance.

Overall, this behavior reflects a principled bias-variance trade-off in covariance estimation rather than a limitation of the proposed framework. Shrinkage regularization proves most beneficial in high-dimensional sparse regimes, while empirical covariance remains appropriate when the underlying feature space is already well-conditioned, thereby demonstrating the adaptability of the proposed approach across diverse data modalities.

Since each covariance matrix $\hat{\Sigma}_j$ obtained from the Gaussian mixture model is positive semi-definite by construction, and $\Sigma_{\mathbf{m}} \succ 0$ due to shrinkage regularisation, the calibrated covariance $\bar{\Sigma}_i$ being expressed as a convex combination of positive semi-definite matrices is itself positive semi-definite. Consequently, the resulting covariance always defines a valid Gaussian distribution, enabling stable sampling and well-conditioned Cholesky decomposition during synthetic sample generation.

Now, the question arises whether the mean of the samples generated in the minority class is a good estimate of the mean of the original distribution of the minority class. Theorem 2 states that the sample mean of the elements sampled from the distribution $\mathcal{N}(\bar{\mu}_j, \bar{\Sigma}_j)$ is the unbiased estimator of the population mean $\bar{\mu}_j$.

Theorem 2. *Let $l'_1, l'_2, \cdots, l'_k$ be the samples generated from $\mathcal{N}(\bar{\mu}_j, \bar{\Sigma}_j)$. Then, the sample mean l' of the generated points l'_i is the unbiased estimator of the population mean $\bar{\mu}_j$.*

Proof. The **proof of Theorem** 2 is provided in the Supplementary section S2.2. □

Remark 2. In Algorithm 2, the samples in the minority class are generated using the statistics of the mean and variance of all the components of the majority classes and the Intermediate Classes. For each datapoint in the minority class, the statistics mean is estimated by considering the mean of the neighboring components and the corresponding datapoint of the particular class using the associated weights. Also, for each datapoint, the statistics covariance is calibrated by considering the covariance of the neighboring component and the covariance of this minority class. Without loss of generality, we assume the case for the sample generation in the cth class. The collection W stores w_i, where the weights w_i associated with the means and covariance are the inverse of the sample size of the particular component $(\hat{\mu}_i, \hat{\Sigma}_i)$ and the weight for the last term (data point of the minority class for the mean calculation and the covariance for the covariance calculation of the minority class) is 1 minus the sum of the rest of the weights. This results in the new mean $(\bar{\mu}_i)$ and new covariance $(\bar{\Sigma}_i)$ associated with each data point. The number of elements to be generated in the cth class is n_c, and the number of elements to be generated corresponding to each element in the cth class is $\hat{n}_c$. Using this new mean and covariance, a set of elements of cardinality n are randomly sampled. $\tilde{L}_{gen}$, $\tilde{Y}_{gen}$ represent the newly generated points and their associated labels corresponding to a particular minority class. The algorithms return the oversampled minority class $\hat{L}$ and its corresponding labels $\hat{Y}$.

3.4. Sparsity-imposing autoencoders with class-preserving latents

When dealing with tabular data, we apply our oversampling algorithm directly to the available features to generate synthetic samples for classes with insufficient data. However, challenges arise when we deal with image data as the dataset is complex and has high dimensionality, and the classes need to be better separated. To overcome this, we use an Autoencoder network, which extracts meaningful latent representation from the data and learns the latent space. Traditional autoencoders excel in learning latent representations from abundant data, but their performance can suffer when faced with imbalanced datasets. This framework addresses this limitation by incorporating novel constraints and a latent classifier network, leading to improved representation quality and generalization on imbalanced data.

To overcome the above challenges, we train our autoencoder so that the classes in the latent space are well separated. Consider our Image space (X) and the corresponding latent space as $(\mathscr{L})$ as a vector space, and let the linear transformation from the Image space to the Latent space be $\mathcal{T} : X \to \mathscr{L}$. Higgins et al. [26] stated the conditions that need to be satisfied to consider a linear transformation as a Group (the definition of which is stated below). A vector representation (X) is disentangled if the decomposition of the vector representation $X = X_1 \times X_2 \cdots X_K$ into several subspaces is such that each subspace is transformed into a unique symmetric transformation $(\mathcal{T}(X_i))$.

Let G be a group with the binary operation $*$ and identity element e. Let X be any set. We define the action $(\gamma : G \times X \mapsto X)$ of the Group G on X as

1. $\gamma(e, x) = x$
2. $\gamma(a, \gamma(b, x)) = \gamma(a * b, x)$

Suppose the latent space $\mathscr{L} \in \mathbb{R}^d$ is a vector space over $\mathbb{R}$, and a symmetric group decomposes into independent subgroups. Higgins et al. [26] proposed that a vector has a disentangled representation if, for a specific decomposition of a symmetric group, the vector decomposition to independent subspaces follows that each subspace is affected by the action of a single group and the remaining subgroups remain unaffected.

Definition 2. Consider the Group Action, $G \times X \to X$, and the direct product of the Group G as the decomposition, $G = G_1 \times G_2 \times \cdots \times G_m$. Let the action of the full group be $*_g$, and the action of each subgroup is $*_{sg}$ respectively. Then, the action is disentangled [27] if the decomposition $X = X_1 \times X_2 \times \cdots \times X_m$, and the actions for each subgroup $*_{sg} : G \times X_i \to X_i, i \in 1, 2, \cdots, n$ satisfies:

$$(g_1, g_2, \cdots, g_m) *_g (x_1, x_2, \cdots, x_m) = (g_1 *_{sg} x_1, g_2 *_{sg} x_2, \cdots, g_m *_{sg} x_m) \quad \forall\ g_i \in G_i,\ \forall\ l_i \in L_i \tag{7}$$

Definition 3. Let X and Y be a group action of $(G, *)$ on X and Y, respectively. If X and Y are both G-sets for the same group G, then a function $f : X \to Y$ is said to be equivariant [27] if.

$$f(g * x) = g * f(x) \qquad \forall\ g \in G \ \ \& \ \ \forall\ x \in X \tag{8}$$

Theorem 3. *Let $f : X \to \mathscr{L}$ be an equivariant map from the images to the latent feature space, and let $\tilde{f} : \mathscr{L} \to Y$ be an equivariant map from the latent feature space to the Reconstructed Image space. Let X, Z, and Y be the G-sets. Then there exists a map $\rho : X \to Y$, $\rho = \tilde{f} o f$ such that ρ satisfies:*

$$\rho(g * x) = g * \rho(x) \qquad \forall\ g \in G \ \ \& \ \ \forall\ x \in X. \tag{9}$$

Proof.

The **proof of Theorem** 3 is provided in the Supplementary section S2.3. □

Remark 3. From Theorem 3, we can infer that the goal of disentanglement is finding an equivariant map ρ which satisfies Eq. (9). Moreover, Autoencoder aims to learn the equivariant map introduced in Definition 3.

Algorithm 2: Pseudocode of Generating Samples in Minority Class(#$Samples < \zeta$).

Input: Imbalanced feature vectors and labels as $\mathscr{L}$ and Y respectively. $\hat{\mu}$, $\hat{\Sigma}$, and S as the mean, covariance, and the sample size corresponding to various components.
Initialize: $\mu_{\mathrm{m}}, \Sigma_{\mathrm{m}}$ as empty.
Output: The Oversampled feature vector $\hat{L}$ and the corresponding labels as $\hat{Y}$.

Compute:
idx $\leftarrow \phi$
flag $\leftarrow 0$
/* For loop running over all the classes. */
for i **from** 1 **to** K **do**
  $L_i, Y_i \leftarrow$ obtains the feature vectors and labels of the ith class from $\mathscr{L}$, Y.
  $n_i = N_1 - N_i$
  if $N_i < \zeta$ **then**
    if $flag == 0$ **then**
      for j from 1 **to** $|S|$ **do**
        $W[j] \leftarrow 1/S_j$ /* The weights are assigned to each component in this loop. */
      end for
      flag $= 1$
    end if
    $\hat{n}_i = n_i/N_i$
    $\mu_{\mathrm{m}}, \Sigma_{\mathrm{m}} \leftarrow$ Gaussian Distribution parameters corresponding to L_i
    /* Loop running over all the elements in the Minority Class. */
    for l in L_i **do**
      Initialize $\hat{\mu}_l$,$\hat{\Sigma}_l$
      $idx \leftarrow$ the index of the k nearest neighbors in $\hat{\mu}$
      Initialize $\bar{\mu}_l$ as 0 and $\bar{\Sigma}_l$ as 0
      /* Loop for calculating the Mean. */
      for j in idx **do**
        $\bar{\mu}_l = \bar{\mu}_l + W[j] \times \hat{\mu}[j]$
      end for
      $w = 1 - \sum_{i=1}^{|S|} W$
      $\bar{\mu}_l = \bar{\mu}_l + w \times l$
      for j in idx **do** /* Loop for calculating the Covariance. */
        $\bar{\Sigma}_l = \bar{\Sigma}_l + W[j] \times \hat{\Sigma}[j]$
      end for
      $\bar{\Sigma}_l = \bar{\Sigma}_l + w \times \Sigma_{\mathrm{m}}$
      /* Loop for generating samples from the estimated Gaussian parameters. */
      for j from 1 to $\hat{n}_i$ **do**
        $\tilde{L}_{gen} \leftarrow$ Randomly drawn sample from a normal distribution with mean and covariance as $\bar{\mu}_l$ and $\bar{\Sigma}_l$ respectively.
      end for
      $\tilde{Y}_{gen} \leftarrow$ array of size $\hat{n}_i$ containing the ith class index.
    end for
    $\hat{L} \leftarrow \tilde{L}_{gen}, \hat{Y} \leftarrow \tilde{Y}_{gen}$
  end if
end for
return $\hat{L}, \hat{Y}$

To produce a disentangled latent vector representation, as defined above, and class-preserving in the latent space, we impose certain constraints while learning the Autoencoder network. These constraints help us obtain well-separated clusters in the latent space. The constraints we will be imposing are as follows:

(a) *Autoencoder Consistency Loss:* Autoencoder Consistency Loss ($\mathcal{L}_{AC}$) is essentially the reconstruction loss. However, since we use latents that are explicitly classwise separated in the latent space, the loss emphasizes reconstructing images to be more similar to those within the same class. This further reduces the chance of generating noisy images, thereby justifying the term Autoencoder Consistency Loss.

(b) *Explicit Between-Class Separation:* We introduce a second constraint term to prevent classes from overlapping in the latent space. This term utilizes the negative dot product between latent vectors from different classes, effectively pushing them apart and promoting discriminative representations. This is achieved by introducing the loss function $\mathcal{L}_{CS}$.

(c) *MLP-based Latent Classification Guidance:* An MLP classifier operates on the latent representations, providing feedback to the autoencoder. This feedback loop guides the autoencoder to learn feature vectors relevant to classification, further enhancing class separation and representation quality, which we achieve by introducing the loss function $\mathcal{L}_{CG}$.

Combining these components, the overall loss function aims to achieve three objectives simultaneously: (i) Faithful reconstruction of the original data (measured using a reconstruction loss function, i.e., Mean Squared Error Loss). (ii) Separation of data points from different classes in the latent space (enforced by the Explicit Between-Class Separation Loss). (iii) Providing supervised guidance, encouraging the encoder to learn a more discriminative and structured latent space (enforced by MLP-based Latent Classification Guidance).

Following the above discussion on training the autoencoder, we used a loss function consisting of three components: the Autoencoder Consistency Loss ($\mathcal{L}_{AC}$), which computes the square of the difference between the ground truth element and the corresponding element in the pre-

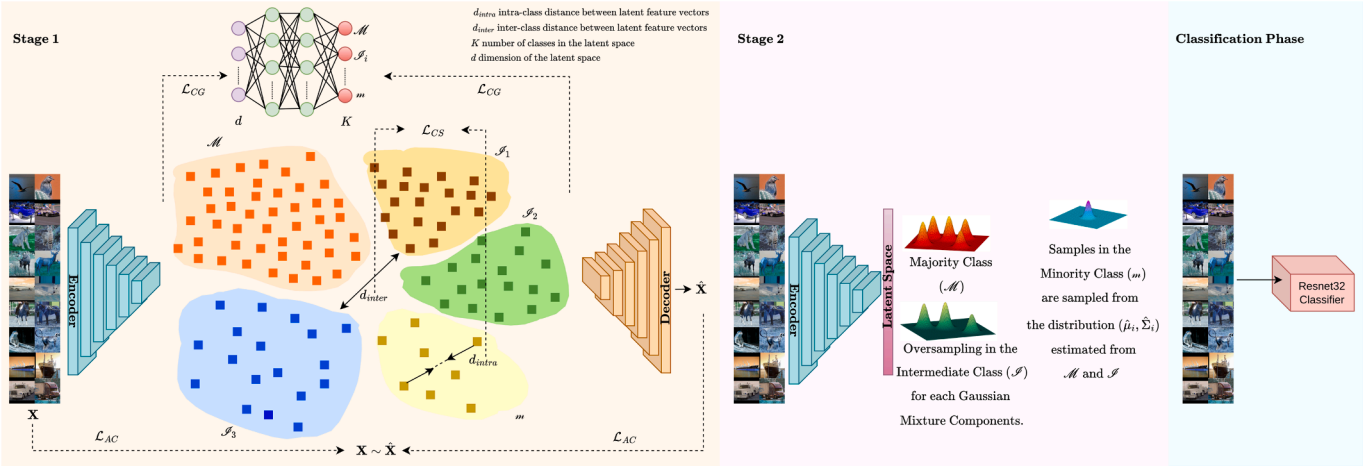


Fig. 3. Illustrations of the proposed RCS framework, Stage 1 depicts the autoencoder training, and Stage 2 portrays the data calibration in the latent space, followed by a classification phase where we employ ResNet32 architecture for Classification.

dicted output vector from the decoder defined in Eq. (10) as:

$$\mathcal{L}_{AC} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2; \tag{10}$$

a latent classifier loss ($\mathcal{L}_{CG}$), which preserves the class labels defined in Eq. (11) as:

$$\mathcal{L}_{CG} = -\left(\sum_{i=1}^{K}[y_i \log(p_i) + (1-y_i)\log(1-p_i)]\right); \tag{11}$$

and the supervised contrastive loss ($\mathcal{L}_{CS}$), based on Khosla et al. (2020) [28], which trains the autoencoder to align feature vectors of the same class in the latent space, defined in Eq. (12) as:

$$\mathcal{L}_{CS} = \sum_{i=1}^{n}\log\left(\frac{-1}{|Q(i)|}\right)\sum_{j\in Q(i)}\frac{exp(l_i.l_j/t)}{\sum_{z\in Z(i)}exp(l_i.l_z/t)}; \tag{12}$$

where, n is the total number of samples, $Q(i) = \{j : 1 < j \leq n \ \& \ y_j = y_i\}$, $Z(i) = \{z : 1 < z \leq n \ \& \ z \neq i\}$ and t is the Temperature hyperparameter. Then the loss for our autoencoder ($\mathcal{L}$) becomes,

$$\mathcal{L} = \mathcal{L}_{AC} + \mathcal{L}_{CG} + \mathcal{L}_{CS}. \tag{13}$$

Traditional autoencoders excel in learning latent representations from abundant data, but their performance can suffer when faced with imbalanced datasets. This framework for our autoencoder aims to address this limitation by incorporating novel constraints and a dynamic classifier network, leading to improved representation quality and generalization on imbalanced data.

The raw images are compressed to a lower-dimensional feature vector in the encoder part of the autoencoder. The latent feature vectors are then passed through the classifier network so that our model can make accurate predictions on the latent space, optimizing the parameters by minimizing the cross-entropy loss ($\mathcal{L}_{CG}$). Additionally, our model learns the feature vectors contrastively using the loss function $\mathcal{L}_{CS}$. The decoder utilizes the lower-dimensional feature vector to reconstruct the image. We calculate the mean square error loss $\mathcal{L}_{AC}$ using the reconstructed and the original image. By minimizing the $\mathcal{L}_{CG}$, $\mathcal{L}_{CS}$, and $\mathcal{L}_{AC}$ losses, we effectively train our autoencoder network. This training approach helps the autoencoder learn latent representations that produce disentangled latent representations for different classes. Such latent representations for different classes do not mix with each other, making them useful for oversampling purposes. Overall framework of autoencoder training shown in the Fig. 3 (Stage 1).

3.5. Putting it all together

After training the Autoencoder, the raw dataset is passed through the Encoder network to obtain the latent feature vectors. These latent feature vectors are then categorized into the Majority class, the Intermediate class, and the Minority class. For the Majority class, we compute the Gaussian Mixture components and their corresponding parameters $(\mu_{\mathscr{M}}, \Sigma_{\mathscr{M}})$. For each Intermediate class $\mathscr{I} \in \hat{\mathscr{I}}$, we derive the Gaussian Mixture components using the distribution parameterized by $(\mu_{\mathscr{I}}, \Sigma_{\mathscr{I}})$, from which the required number of synthetic samples are drawn to balance the Intermediate class.

For each sample in the Minority class $\mathfrak{m} \in \hat{\mathfrak{m}}$, we estimate the Gaussian parameters based on a combination of the Gaussian components from the Intermediate classes $(\mu_{\hat{\mathscr{I}}}, \Sigma_{\hat{\mathscr{I}}})$ and those from the Majority class $(\mu_{\mathscr{M}}, \Sigma_{\mathscr{M}})$. Synthetic samples for the Minority class are then generated using these estimated Gaussian parameters. We sample synthetic data points in the latent (or feature) space using the estimated parameters. For image data, we pass the oversampled latent (or feature) vectors through the Decoder Network to retrieve the synthetic images. We use the oversampled feature vectors for tabular data to train the classification network. Fig. 3 shows the overview of the proposed method.

3.6. Computational complexity

The computational complexity of the proposed RCS framework is analyzed by decomposing it into its constituent steps. Throughout the analysis, we assume worst-case complexity [13,29].

Step 1: Disentangled Autoencoder Training

Each epoch consists of one forward pass(encoder and decoder), one backward pass (gradient computation), supervised contrastive loss, latent classifier loss. All operations are linear in the number of samples. Assuming the encoder-decoder architecture has bounded depth and width proportional to the latent dimension d, the per-sample forward-backward cost scales as $O(d)$. Latent losses (dot products, softmax): $O(d)$. Hence, per sample we have complexity of $O(d)$ So, if the number of epochs is E, we have $O(E.N.d)$.

Step 2: Gaussian Mixture Model (GMM) Estimation

Each majority or intermediate class is modeled using a GMM. So we have Expectation-Maximization (EM) Complexity. For a GMM with ξ_c components we have E-step and M-step.

E-step: For each sample, compute likelihood against each component: Mahalanobis distance: $O(d^2)$, Determinant/inverse once per iteration. Hence we have the complexity of $O(N_c\xi_c d^2)$ M-step:

Mean Update: $O(N_c.d)$, Covariance Update: $O(N_c.d^2)$ So we obtain $O(N_c.\xi_c.d^2)$. Now if we have $\mathscr{G}$ many EM iterations, we obtain the

Table 2
Summary of the Image Datasets.

Dataset	Shape	Classes	IR	Training Set (For each Class)	Testing Set (For each class)
MNIST	28×28×1	10	100	4000, 2000, 1000, 750, 500, 350, 200, 100, 60, 40	1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000
FMNIST	28×28×1	10	100	4000, 2000, 1000, 750, 500, 350, 200, 100, 60, 40	1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000
SVHN	32×32×3	10	56.25	4500, 2000, 1000, 800, 600, 500, 400, 250, 150, 80	1744, 5099, 4149, 2882, 2523, 2384, 1977, 2019, 1660, 1595
CIFAR-10	32×32×3	10	56.25	4500, 2000, 1000, 800, 600, 500, 400, 250, 150, 80	1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1002
STL10	32×32×3	10	12.5	500, 400, 350, 300, 250, 200, 150, 100, 80, 40	100, 100, 100, 100, 100, 100, 100, 100, 100, 100
GTSRB	32×32×3	43	10.71	2250, 2220, 2160, ⋯, 210	750, 450, ⋯, 90, 60
SUN397	32×32×3 & 64×64×3	50	11.58	938, 892, 708, 675, 563, ⋯, 85, 82, 81, 81	219, 208, 165, 158, 132, ⋯, 19, 19, 19, 19

Table 3
Summary of the Tabular Datasets.

Dataset	Features	Classes	IR	Training Set (For each Class for all Splits)	Testing Set (For each Class for all Splits)
Optical_digits	64	2	9.13	4052, 444	1014, 110
Webpage	300	2	34.44	27039, 785	6760, 196
Isolet	617	2	11.99	5757, 480	1440, 120
Contraceptive	8	3	1.89	502, 408, 266	126, 102, 67
Dermatology	34	6	2.32	88, 57, 47, 38	22, 12, 14, 10
Wine	13	3	1.43	50, 42, 35	13, 11, 8

complexity as $O(\mathcal{G}.N_c.\xi_c.d^2)$. Summing over all the classes we obtain $\sum_{c=1}^{K} O(\mathcal{G}.N_c.\xi_c.d^2) = O(\mathcal{G}.d^2 \sum_{c=1}^{K} .N_c.\xi_c)$. So from here we obtain the total GMM Complexity as $O(\mathcal{G}.d^2.N.\xi)$, since $\sum N_c = N$ and $\sum \xi_c = \xi$.

Step 3: k-Nearest Component Search

For each minority sample $l \in \mathbb{R}^d$, distance to each GMM mean $O(d)$ and total components ξ. For search we have complexity $O(\xi.d)$. For all minority sample $N_m \leq N$ we have $O(N_m.\xi.d)$ Since $N_m \leq N$, we have total complexity of $O(N.\xi.d)$.

Step 4: Mean and Covariance Calibration

For each minority sample from mean calibration in Equation (3), vector addition and scaling has O(d), repeated k times $O(k.d)$. From the covariance calibration in Equation (4), matrix addition has complexity as $O(d^2)$ which is repeated k times so we obtain $O(k.d^2)$. So the total calibration cost for all minority samples $O(N_m.k.d^2)$.

Step 5: Synthetic sample generation

Sampling from a multivariate Gaussian requires a Cholesky decomposition with complexity $O(d^3)$ per distinct calibrated covariance, followed by $O(d^2)$ per sample generation. Since each calibrated covariance is reused to generate multiple samples, the amortized sampling cost is $O(N_m d^2)$, with a one-time preprocessing cost bounded by $O(N_m d^3)$ in the worst case.

So the overall complexity of RCS becomes $O(N.d) + O(\mathcal{G}.d^2.N.\xi) + O(N.\xi.d) + O(N_m.k.d^2) + O(N_m.d^3) = O(N.d) + O(\mathcal{G}.d^2.N.\xi)$

Since $\mathcal{G} \geq 1$, $d \ll N\xi$ in practical settings, and sampling costs are amortized across reused covariances, the overall complexity is dominated by the GMM estimation step.

Therefore, the overall computational complexity of RCS is dominated by the GMM estimation stage and can be expressed as

$$O(\mathcal{G}\, N\, \xi\, d^2),$$

with lower-order terms arising from representation learning, neighbor search, calibration, and amortized sampling.

4. Experiment and discussions

4.1. Dataset overview

We evaluate on seven benchmark datasets. Following [7], we use MNIST, FMNIST, CIFAR-10, SVHN, and SUN397, maintaining the same number of classes, imbalance ratios, and image sizes: MNIST and FMNIST (10 classes, 28×28) with imbalanced training sets of 4000-40 samples per class; CIFAR-10 and SVHN (10 classes, 32×32) with 4500-80 samples per class; and SUN397, where 50 classes were selected and images scaled to 32×32 and 64×64 with an imbalance ratio (IR) of 14.21. In addition, we include GTSRB [30] (43 classes, scaled to 32×32, IR = 10.71) and STL-10 [31] (3 channels, 32×32, IR = 12.5). Dataset details are provided in Table 2.

For the tabular dataset, we considered datasets from the UCI Machine Learning repository: Optical_digits, Webpage, Isolet, Contraceptive, Dermatology, Thyroid, and Wine. Optical_digits, Webpage, and Isolet are datasets with binary classes, whereas Contraceptive, Dermatology, and Wine have multi-class data with 3, 6, and 3 classes, respectively. Detailed descriptions of these datasets are provided in Table 3.

For **textual datasets**, we used Spam or Ham, Spam or Not Spam, Emotion, and Hate Speech datasets with detailed descriptions in Supplementary Section S3.1.

4.2. Training overview

As described above we used the Encoder-Decoder architecture. The Encoder network consisted of two Convolution Layers, two Batch Normalization layers, and a fully connected layer to obtain the Latent vector with dimensions 300 for the single-channel image and 600 for the multi-channel image. The Decoder network consisted of the deconvolution layers and the batch normalization layers. The Latent Classifier Network consisted of a Multi-layered Perceptron with 4 hidden layers. We used the Adam optimizer with a learning rate of $1e-4$. The Autoencoder was trained over 200 epochs using the loss function $\mathcal{L}$.

For the sample generation process, we generated the samples in two phases. In the first phase, we generated the samples by traversing through each class, and the number of samples in each class was greater than ζ were oversampled using Algorithm 1. In the second phase, we similarly traversed through each class, and if the number of samples in each class was less than ζ, we oversampled those classes using Algorithm 2. After including the synthetic data points, we checked the performance of the oversampled data using the ResNet32 network. For image datasets, **the optimal hyperparameter values** are reported in Table S1 (Supplementary).

For the Tabular data, the network was trained with Cross-entropy loss with the Adam optimizer, with a learning rate of 0.0001 and a number of epochs of 100. For classification of the oversampled feature vectors, we used a Multi-layered Perceptron, trained over 100 epochs with Cross-entropy loss with Adam optimizer, with a learning rate of $1e-3$.

Table 4
Results for MNIST and FMNIST.

Methods	Venue	MNIST				FMNIST			
		BACC	MCC	F1-Score	Gmean	BACC	MCC	F1-Score	Gmean
Baseline		0.923±0.008	0.914±0.009	0.921±0.008	0.956±0.005	0.769±0.004	0.747±0.004	0.757±0.006	0.865±0.002
ROS		0.89±0.003	0.889±0.026	0.89±0.008	0.895±0.027	0.804±0.003	0.783±0.004	0.802±0.003	0.792±0.003
SMOTE	[JAIR'2002]	0.932±0.009	0.925±0.01	0.931±0.009	0.962±0.005	0.782±0.007	0.761±0.007	0.77±0.01	0.873±0.004
cDCGAN	[ICLR'2016]	0.943±0.004	0.937±0.005	0.942±0.005	0.968±0.003	0.788±0.007	0.769±0.009	0.779±0.005	0.877±0.004
GAMO	[ICCV'2019]	0.873±0.008	0.861±0.008	0.869±0.009	0.928±0.005	0.841±0.011	0.825±0.012	0.835±0.011	0.909±0.006
TailCalibX	[ICLR'2021]	0.807±0.016	0.788±0.018	0.795±0.015	0.889±0.009	0.69±0.019	0.66±0.021	0.672±0.018	0.816±0.012
DeepSMOTE	[TNNLS'2022]	0.943±0.005	0.937±0.005	0.942±0.005	0.968±0.003	0.802±0.011	0.783±0.012	0.793±0.01	0.885±0.006
DGCMM	[TPAMI'2023]	0.945±0.002	0.939±0.002	0.943±0.002	0.969±0.001	0.819±0.003	0.803±0.003	0.812±0.003	0.896±0.002
RCS		**0.962±0.002**	**0.958±0.002**	**0.962±0.002**	**0.979±0.001**	**0.845±0.004**	**0.829±0.004**	**0.84±0.006**	**0.911±0.002**

Table 5
Results for CIFAR10 and SVHN.

Methods	Venue	CIFAR10				SVHN			
		BACC	MCC	F1-Score	Gmean	BACC	MCC	F1-Score	Gmean
Baseline		0.377±0.006	0.316±0.007	0.334±0.015	0.592±0.005	0.624±0.016	0.588±0.018	0.608±0.015	0.773±0.011
ROS		0.235±0.009	0.155±0.009	0.203±0.02	0.463±0.009	0.543±0.027	0.504±0.029	0.507±0.027	0.718±0.019
SMOTE	[JAIR'2002]	0.371±0.013	0.309±0.013	0.332±0.028	0.587±0.011	0.617±0.017	0.58±0.018	0.601±0.019	0.769±0.011
cDCGAN	[ICLR'2016]	0.395±0.021	0.337±0.024	0.358±0.018	0.607±0.017	0.701±0.039	0.672±0.043	0.686±0.045	0.823±0.025
GAMO	[ICCV'2019]	0.43±0.012	0.375±0.014	0.402±0.012	0.635±0.009	0.719±0.016	0.691±0.017	0.714±0.016	0.835±0.01
TailCalibX	[ICLR'2021]	0.355±0.015	0.294±0.018	0.302±0.022	0.574±0.013	0.528±0.021	0.481±0.024	0.501±0.021	0.707±0.015
DeepSMOTE	[TNNLS'2022]	0.43±0.007	0.373±0.008	0.411±0.012	0.635±0.006	0.691±0.033	0.662±0.035	0.678±0.039	0.817±0.021
DGCMM	[TPAMI'2023]	0.432±0.004	0.38±0.004	0.397±0.007	0.636±0.003	0.764±0.009	0.742±0.01	0.762±0.009	0.863±0.006
RCS		**0.472±0.011**	**0.418±0.012**	**0.467±0.011**	**0.667±0.008**	**0.776±0.011**	**0.753±0.012**	**0.773±0.012**	**0.870±0.007**

Table 6
Results for GTSRB and STL10.

Methods	Venue	GTSRB				STL10			
		BACC	MCC	F1-Score	Gmean	BACC	MCC	F1-Score	Gmean
Baseline		0.861±0.012	0.907±0.010	0.866±0.011	0.927±0.007	0.347±0.014	0.280±0.016	0.315±0.016	0.567±0.012
ROS		0.905±0.015	0.939±0.007	0.913±0.013	0.95±0.008	0.342±0.014	0.273±0.016	0.318±0.02	0.563±0.012
SMOTE	[JAIR'2002]	0.883±0.018	0.916±0.017	0.884±0.016	0.939±0.010	0.349±0.000	0.281±0.000	0.323±0.000	0.569±0.000
cDCGAN	[ICLR'2016]	0.878±0.013	0.924±0.008	0.885±0.012	0.936±0.007	0.349±0.006	0.282±0.008	0.324±0.006	0.569±0.005
GAMO	[ICCV'2019]	0.832±0.019	0.878±0.011	0.829±0.015	0.911±0.010	0.391±0.011	0.329±0.009	0.369±0.013	0.604±0.009
TailCalibX	[ICLR'2021]	0.884±0.028	0.919±0.018	0.883±0.027	0.939±0.015	0.419±0.012	0.358±0.014	0.394±0.012	0.626±0.009
DeepSMOTE	[TNNLS'2022]	0.830±0.017	0.879±0.011	0.830±0.015	0.910±0.010	0.369±0.004	0.304±0.005	0.347±0.006	0.586±0.003
DGCMM	[TPAMI'2023]	0.937±0.005	0.956±0.002	0.94±0.004	0.967±0.002	0.390±0.001	0.33±0.002	0.352±0.001	0.603±0.001
RCS		**0.955±0.006**	**0.965±0.004**	**0.948±0.007**	**0.977±0.003**	**0.420±0.009**	**0.360±0.010**	**0.401±0.008**	**0.627±0.007**

Table 7
Results for SUN397 with images of sizes 32×32 and 64×64 respectively.

Methods	Venue	SUN397 with Image Size 32×32				SUN397 with Image Size 64×64			
		BACC	MCC	F1-Score	Gmean	BACC	MCC	F1-Score	Gmean
Baseline		0.237±0.004	0.299±0.004	0.227±0.005	0.484±0.004	0.342±0.018	0.397±0.017	0.338±0.019	0.581±0.015
ROS		0.232±0.011	0.212±0.013	0.200±0.013	0.478±0.011	0.342±0.006	0.331±0.008	0.313±0.004	0.581±0.005
SMOTE	[JAIR'2002]	0.242±0.014	0.299±0.012	0.232±0.018	0.489±0.014	0.347±0.008	0.401±0.009	0.345±0.01	0.586±0.006
cDCGAN	[ICLR'2016]	0.236±0.005	0.268±0.007	0.229±0.008	0.482±0.005	0.294±0.012	0.334±0.011	0.284±0.014	0.538±0.011
GAMO	[ICCV'2019]	0.261±0.007	0.296±0.008	0.263±0.008	0.507±0.007	0.283±0.006	0.333±0.005	0.284±0.005	0.528±0.006
TailCalibX	[ICLR'2021]	0.271±0.011	0.326±0.016	0.255±0.014	0.517±0.01	0.349±0.007	0.414±0.009	0.337±0.004	0.587±0.006
DeepSMOTE	[TNNLS'2022]	0.286±0.008	0.294±0.007	0.27±0.003	0.531±0.007	0.373±0.013	0.394±0.023	0.362±0.016	0.607±0.01
DGCMM	[TPAMI'2023]	0.276±0.002	0.313±0.006	0.275±0.002	0.521±0.002	0.376±0.004	0.421±0.003	0.369±0.004	0.610±0.003
RCS		**0.318±0.009**	**0.330±0.014**	**0.303±0.013**	**0.560±0.008**	**0.406±0.007**	**0.439±0.009**	**0.397±0.004**	**0.633±0.006**

For tabular datasets, **the optimal hyperparameters** are reported in Table S2 (Supplementary).

4.3. Evaluation metrics used

To evaluate the performance of imbalance classification, four well-known metrics were used: Balanced Accuracy (BACC [1]): average recall across all classes; Matthews Correlation Coefficient (MCC [2]): a balanced measure considering true positives, true negatives, false positives, and false negatives; F1-Score [3]: combined precision and recall into a single balanced measure; and Geometric Mean (Gmean [4]): geometric average

[1] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.balanced_accuracy_score.html

[2] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.matthews_corrcoef.html

[3] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

[4] https://imbalanced-learn.org/stable/references/generated/imblearn.metrics.geometric_mean_score.html

of class-specific performance metrics. For all these metrics, higher values indicated better performance.

4.4. Methods used for comparison

For the comparison of the proposed RCS framework on the Image datasets, we used the following algorithms. We considered the Baseline method (ResNet32 trained using the cross-entropy loss function without applying any imbalance handling techniques), Resampling methods such as Random Oversampling(ROS), SMOTE [3], Generative based methods such as Conditional DCGAN (cDCGAN) [19], Generative Adversarial Minority Oversampling (GAMO) [7], DeepSMOTE [9], Deep Generative Mixture Model for Robust Imbalance Classification (DGCMM) [10] and Calibration based method TailCalibX [23]. In the TailCalibX method, oversampling was performed in the latent feature space, followed by decoding the synthesized samples back to the input space using a decoder. For the SMOTE oversampling algorithm, features are first extracted using the ResNet32 classifier network, after which the SMOTE algorithm is applied to the extracted feature vectors.

For tabular and textual datasets, the peer methods used for comparison included Baseline(BL), SMOTE(SM) [3], ADASYN (ADA) [18], BorderlineSMOTE (blSM) [17], and SVMSMOTE (svmSM) [32], ClusterSMOTE (clSM) [33], CURE-SMOTE (cuSM) [34], DBSMOTE (dbSM) [35], Kmeans SMOTE (kmSM) [36], MeanShiftSMOTE (msSM) [37], MWMOTE (mwSM) [38], SOICJ [39], CE-SMOTE (ceSM) [40].

4.5. Results and discussions

In this section, we outline the evaluation procedure and report results on image and tabular datasets, with **textual dataset details and results** provided in Supplementary Section S3.2 and Section S3.4, respectively.

4.5.1. Evaluation procedure

When dealing with the image datasets, we repeated the entire training process in Subsection 4.2 for five different seeds, where the training set is imbalanced, and the test set is balanced. We obtained the mean and the standard deviation on receiving the results corresponding to these seeds. For the tabular and text datasets we fixed the seed value and used stratified 5-fold cross-validation to train the models, compute the metric values, and report the mean and standard deviation across all splits.

4.5.2. Discussion on image dataset

Discussion on MNIST and FMNIST. As shown in Table 4, the proposed RCS framework consistently outperforms all other methods across all evaluation metrics. On the MNIST dataset, the proposed RCS framework achieves a 1.9% improvement in Balanced Accuracy compared to DGCMM, the best-performing state-of-the-art method. Additionally, the MCC, F1-Score, and G-mean improve by 1.9%, 1.9%, and 1%, respectively, over DGCMM. For the FMNIST dataset, the proposed RCS framework outperforms GAMO-the strongest baseline-by 0.4% in Balanced Accuracy, while achieving gains of 0.4%, 0.5%, and 0.2% in MCC, F1-Score, and G-mean, respectively. Compared to MNIST, FMNIST presents more complex patterns, as clothing items exhibit higher intra-class variation and are less distinct from the background. Among the baselines, TailCalibX performs poorly on both datasets due to its inability to capture class distributions in single-channel images. In contrast, the proposed RCS framework effectively addresses this limitation by modeling the local distribution within each class. Since the MNIST dataset is simple, it helps demonstrate model interpretability and explainability techniques. Compared to MNIST, FMNIST presents more complex patterns, as clothing items exhibit higher intra-class variation and are less distinct from the background.

Discussion on CIFAR10, SVHN. Table 5 presents the performance metrics of various state-of-the-art methods and the proposed RCS framework on these datasets. Despite having three channels, CIFAR-10 remains challenging due to the small size of the images, which makes it harder for models to capture fine details. Due to the overlap between various classes, any models trained on this dataset fail to distinguish more minor differences. The SVHN dataset requires careful preprocessing, which includes normalization, cropping, and the challenges with overlapping digits. Our Autoencoder network was trained to overcome this preprocessing overhead so that the classes are well separated in the latent space. Conditional DCGAN performed worse than the proposed RCS framework because of the nature of images in the SVHN dataset. For CIFAR10, the balanced accuracy of our algorithm improved by 4% compared to DGCMM, which is the best among the competitor methods; also the MCC and Gmean increased by 3.8%, 3.1% respectively to the DGCMM method and the F1-Score metric for the proposed RCS framework has increased by 5.6%. For the SVHN dataset, we improved by 1.2% over DGCMM, which is the best-performing state-of-the-art method. The MCC, F1-Score, and Gmean metrics for RCS on SVHN increased by 1.1%, 1.1%, and 0.7% respectively than the DGCMM method.

Discussion on GTSRB and STL10. The experiments on the GTSRB dataset evaluated model performance under limited training/validation samples and a relatively low class-imbalance ratio. The dataset contains traffic sign images captured under varied lighting and weather conditions. As shown in Table 6, the proposed RCS framework outperforms all state-of-the-art baselines: it improves balanced accuracy by 1.8% over DGCMM (the strongest baseline) and by 7.1% over TailCalibX. In addition, MCC, F1-Score, and Gmean increase by 0.9%, 0.8%, and 1% respectively compared to DGCMM. The STL10 dataset presents substantial intra-class variability and inter-class similarity, making classification particularly challenging. Models trained here must effectively capture subtle differences between classes. As shown in Table 6, the proposed RCS framework again achieves the best performance across all metrics. Balanced accuracy improves by 0.1% compared to TailCalibX, while MCC, F1-Score, and Gmean increase by 0.2%, 0.7%, and 0.1% respectively. Against generative models, the improvements are substantial: relative to DGCMM, gains of 3%, 3%, 4.9%, and 2.4% in balanced accuracy, MCC, F1-Score, and Gmean are observed; relative to GAMO, the improvements are 2.9%, 3.1%, 3.2%, and 2.3%. These results highlight that our distribution calibration method consistently surpasses both calibration and generation based approaches. Even in scenarios with lower imbalance, it produces new samples that faithfully reflect the minority-class distribution without distorting the original data.

Discussion on SUN397. SUN397 involves scene recognition that understands the image's context, spatial relationships, and background elements. Table 7 shows that the proposed RCS framework outperforms all other competitive methods. When comparing SUN397 with an image size of 32×32, it is observed that the balanced accuracy metric for the proposed RCS framework improved by 3.2% over the DeepSMOTE method, the best-performing state-of-the-art method. Additionally, the MCC and F-Score increased by 1.7% and 2.8%, respectively, compared to the DGCMM method. The F1-Score metric for the proposed RCS framework has increased by 2.9% over the DeepSMOTE method. On the other hand, when comparing the results for the SUN397 dataset with an image size of 64×64, it is observed that the balanced accuracy value for the proposed RCS framework has improved by 3% over the DGCMM method, which is the best among the state-of-the-art methods. Increasing the image size from 32×32 to 64×64 captures finer details of the images in the dataset, resulting in an 8.8% increase in balanced accuracy. The MCC, F1-Score, and Gmean metrics for RCS on SUN397 escalate by 1.8%, 2.8%, and 2.3%, respectively, compared to the DGCMM method.

Table 8
Results of the proposed RCS Distribution Calibration Method on Tabular Datasets.

Datasets	Metrics	BL	SM	ADA	blSM	clSM	cuSM	DBSM	kmSM	msSM	mwSM	SOICJ	svmSM	csSM	ISMOTE	RCS
Contraceptive	BACC	0.511 ±0.041	0.514 ±0.036	0.501 ±0.038	0.52 ±0.058	0.519 ±0.059	0.486 ±0.053	0.523 ±0.038	0.489 ±0.014	0.493 ±0.05	0.533 ±0.038	0.471 ±0.051	0.507 ±0.049	0.517 ±0.05	0.476 ±0.053	**0.555 ±0.032**
	MCC	0.259 ±0.058	0.258 ±0.046	0.256 ±0.055	0.268 ±0.085	0.268 ±0.084	0.217 ±0.081	0.277 ±0.053	0.229 ±0.024	0.237 ±0.068	0.29 ±0.055	0.217 ±0.065	0.25 ±0.075	0.268 ±0.076	0.205 ±0.079	**0.329 ±0.039**
	F1-Score	0.5 ±0.039	0.497 ±0.027	0.514 ±0.027	0.498 ±0.056	0.496 ±0.059	0.471 ±0.048	0.521 ±0.031	0.489 ±0.02	0.427 ±0.084	0.519 ±0.038	0.463 ±0.068	0.494 ±0.058	0.498 ±0.042	0.430 ±0.069	**0.546 ±0.026**
	Gmean	0.62 ±0.033	0.622 ±0.028	0.613 ±0.031	0.626 ±0.047	0.626 ±0.048	0.599 ±0.044	**0.63** ±0.03	0.603 ±0.012	0.604 ±0.041	0.638 ±0.031	0.589 ±0.041	0.617 ±0.04	0.624 ±0.04	0.368 ±0.211	0.548 ±0.034
Isolet	BACC	0.5 ±0	0.98 ±0.008	0.981 ±0.007	0.978 ±0.008	0.98 ±0.009	0.975 ±0.006	0.978 ±0.006	0.975 ±0.006	0.98 ±0.008	0.979 ±0.008	0.597 ±0.216	0.982 ±0.007	0.981 ±0.008	0.958 ±0.017	**0.997 ±0.005**
	MCC	0 ±0	0.89 ±0.04	0.872 ±0.051	0.845 ±0.038	0.864 ±0.064	0.858 ±0.035	0.868 ±0.059	0.867 ±0.038	0.878 ±0.049	0.885 ±0.035	0.178 ±0.399	0.865 ±0.057	0.874 ±0.063	0.848 ±0.043	**0.976 ±0.041**
	F1-Score	0.886 ±0	0.983 ±0.007	0.979 ±0.01	0.975 ±0.007	0.978 ±0.012	0.977 ±0.006	0.979 ±0.012	0.979 ±0.007	0.981 ±0.009	0.982 ±0.006	0.906 ±0.043	0.978 ±0.011	0.98 ±0.012	0.921 ±0.024	**0.996 ±0.006**
	Gmean	0.5 ±0	0.98 ±0.008	0.981 ±0.007	0.978 ±0.008	0.98 ±0.009	0.975 ±0.006	0.978 ±0.006	0.975 ±0.006	0.98 ±0.008	0.979 ±0.008	0.597 ±0.216	0.982 ±0.007	0.981 ±0.008	0.958 ±0.017	**0.997 ±0.005**
Webpage	BACC	0.864 ±0.013	0.93 ±0.016	0.922 ±0.016	0.913 ±0.018	0.922 ±0.01	0.915 ±0.005	0.925 ±0.014	0.871 ±0.013	0.929 ±0.013	0.925 ±0.015	0.878 ±0.016	0.923 ±0.012	0.925 ±0.022	0.914 ±0.016	**0.938 ±0.022**
	MCC	0.768 ±0.021	0.647 ±0.022	0.658 ±0.049	0.676 ±0.042	0.709 ±0.02	0.76 ±0.019	0.643 ±0.044	0.776 ±0.014	0.613 ±0.01	0.68 ±0.045	0.758 ±0.019	0.644 ±0.033	0.667 ±0.026	0.682 ±0.017	**0.879 ±0.049**
	F1-Score	0.988 ±0.001	0.975 ±0.003	0.976 ±0.005	0.979 ±0.004	0.982 ±0.002	0.986 ±0.001	0.974 ±0.005	0.988 ±0.001	0.971 ±0.001	0.978 ±0.005	0.987 ±0.001	0.975 ±0.004	0.977 ±0.004	0.833 ±0.010	**0.993 ±0.003**
	Gmean	0.864 ±0.013	0.93 ±0.016	0.922 ±0.016	0.913 ±0.018	0.922 ±0.01	0.915 ±0.005	0.925 ±0.014	0.871 ±0.013	0.929 ±0.013	0.925 ±0.015	0.878 ±0.016	0.923 ±0.012	0.925 ±0.022	0.911 ±0.017	**0.936 ±0.024**
Optical_digits	BACC	0.5 ±0	0.971 ±0.017	0.975 ±0.006	0.969 ±0.009	0.977 ±0.01	0.967 ±0.016	0.977 ±0.008	0.928 ±0.028	0.974 ±0.004	0.977 ±0.009	0.86 ±0.202	0.983 ±0.006	0.974 ±0.008	0.933 ±0.008	**0.991 ±0.004**
	MCC	0 ±0	0.895 ±0.047	0.922 ±0.023	0.931 ±0.015	0.935 ±0.026	0.934 ±0.023	0.925 ±0.028	0.852 ±0.071	0.929 ±0.026	0.937 ±0.025	0.727 ±0.407	0.914 ±0.022	0.922 ±0.034	0.743 ±0.018	**0.942 ±0.025**
	F1-Score	0.855 ±0.001	0.98 ±0.009	0.986 ±0.004	0.988 ±0.003	0.988 ±0.005	0.988 ±0.004	0.986 ±0.005	0.973 ±0.013	0.987 ±0.005	0.989 ±0.005	0.958 ±0.058	0.984 ±0.004	0.986 ±0.006	0.863 ±0.012	**0.989 ±0.005**
	Gmean	0.5 ±0	0.971 ±0.017	0.975 ±0.006	0.969 ±0.009	0.977 ±0.01	0.967 ±0.016	0.977 ±0.008	0.928 ±0.028	0.974 ±0.004	0.977 ±0.009	0.86 ±0.202	0.983 ±0.006	0.974 ±0.008	0.932 ±0.008	**0.991 ±0.004**
Dermatology	BACC	0.782 ±0.083	0.97 ±0.026	0.854 ±0.029	0.899 ±0.071	0.968 ±0.03	0.798 ±0.028	0.977 ±0.015	0.97 ±0.017	0.971 ±0.027	0.975 ±0.024	0.981 ±0.022	0.979 ±0.014	0.942 ±0.067	0.862 ±0.038	**0.981 ±0.025**
	MCC	0.837 ±0.045	0.966 ±0.031	0.839 ±0.033	0.91 ±0.035	0.965 ±0.033	0.893 ±0.032	0.976 ±0.015	0.969 ±0.014	0.966 ±0.034	0.976 ±0.023	0.979 ±0.023	0.975 ±0.02	0.94 ±0.064	0.873 ±0.035	**0.979 ±0.029**
	F1-Score	0.818 ±0.062	0.971 ±0.027	0.815 ±0.055	0.921 ±0.037	0.972 ±0.026	0.888 ±0.025	0.98 ±0.013	0.974 ±0.013	0.972 ±0.028	0.98 ±0.02	0.983 ±0.019	0.98 ±0.016	0.939 ±0.079	0.833 ±0.058	**0.983 ±0.023**
	Gmean	0.871 ±0.05	0.982 ±0.016	0.911 ±0.018	0.941 ±0.04	0.981 ±0.018	0.885 ±0.017	0.987 ±0.009	0.982 ±0.01	0.983 ±0.016	0.986 ±0.014	0.989 ±0.013	0.988 ±0.009	0.965 ±0.04	0.471 ±0.437	**0.98 ±0.027**
Wine	BACC	0.759 ±0.142	0.795 ±0.128	0.787 ±0.071	0.827 ±0.187	0.731 ±0.118	0.884 ±0.087	0.874 ±0.122	0.781 ±0.129	0.655 ±0.077	0.722 ±0.07	0.782 ±0.123	0.744 ±0.217	0.677 ±0.188	0.770 ±0.134	**0.95 ±0.039**
	MCC	0.695 ±0.163	0.711 ±0.165	0.683 ±0.103	0.763 ±0.25	0.623 ±0.172	0.822 ±0.123	0.807 ±0.172	0.699 ±0.185	0.548 ±0.096	0.609 ±0.066	0.693 ±0.156	0.651 ±0.283	0.562 ±0.234	0.670 ±0.189	**0.92 ±0.063**
	F1-Score	0.738 ±0.166	0.716 ±0.195	0.708 ±0.151	0.794 ±0.226	0.61 ±0.205	0.851 ±0.113	0.815 ±0.219	0.683 ±0.227	0.553 ±0.059	0.621 ±0.101	0.692 ±0.205	0.669 ±0.289	0.541 ±0.252	0.682 ±0.190	**0.943 ±0.048**
	Gmean	0.815 ±0.109	0.837 ±0.103	0.83 ±0.058	0.864 ±0.147	0.786 ±0.095	0.908 ±0.069	0.899 ±0.099	0.826 ±0.106	0.731 ±0.053	0.779 ±0.054	0.825 ±0.101	0.799 ±0.171	0.743 ±0.148	0.530 ±0.365	**0.946 ±0.044**

4.5.3. Discussion on tabular datasets

We performed experiments on 7 tabular datasets: Optical_digits, Webpage, Isolet, Contraceptive, Dermatology, Thyroid, and Wine. The feature vectors were directly used for our calibration technique. We split the feature vectors and their corresponding labels into 5 parts in each of these datasets using the 5-fold Stratified cross-validation technique. Then, in each of these 5 folds, we used 4 parts for training and the remaining 1 for validation. For the Optical_digits dataset, our model handled variations in handwriting styles, and it beat the balanced accuracy metric of the highest competitive method, i.e., svmSM (SVM SMOTE) by 0.8%. To check the performance of our algorithm, we consider the method with the second-highest accuracy and compare it with the proposed RCS framework. On the Contraceptive dataset, the metrics balanced accuracy, MCC, and F1-Score increased by $2.2\%, 3.9\%$, and 2.7% than the mwSM method. Moreover, for the Contraceptive dataset, the GMean decreased by 1.5%, falling below the performance of DBSM, the current best-performing method.

Table 9
Figures depicting the TSNE-plot of the latent feature vectors from the DeepSMOTE Autoencoder and our Autoencoder for the imbalanced feature vectors and the feature vectors after applying the oversampling algorithm.

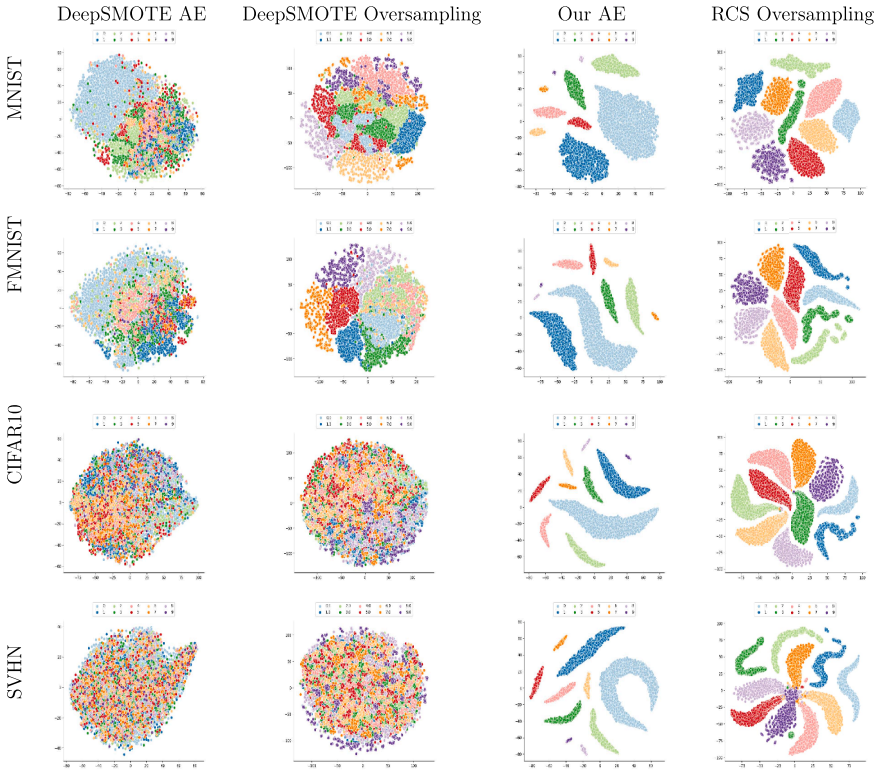
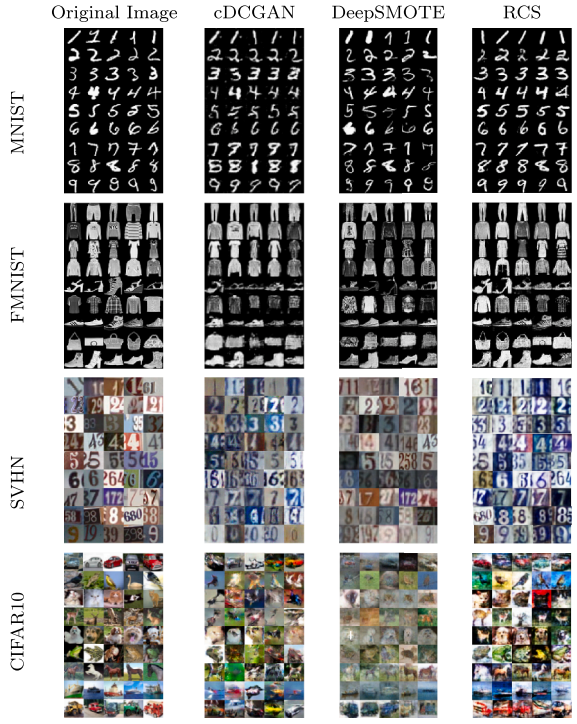


Table 10
Figures depicting the Synthetic Image Generation of Various Methods and The Proposed RCS Framework for MNIST, FMNIST, SVHN, and CIFAR10.



For the Isolet dataset, we have two best-performing state-of-the-art methods, as we can observe in Table 8 that the metrics MCC and F1-Score increase by 8.6% and 1.3% respectively and the metrics BACC and Gmean gain by 1.3% and 1.5% respectively than SVM SMOTE. For the Dermatology dataset, the balanced accuracy, F1-Score, and MCC metrics are equal to the RCS method for the SOICJ dataset. Also, we can observe that for the RCS method, the Gmean metric value is lower than the SOICJ method by 0.9%. For the Wine dataset, the balanced accuracy, MCC, F1-Score, and Gmean metric values have increased by 6.6%, 9.8%, 9.2%, and 3.8% than Cure SMOTE, which is the best performing state-of-the-art method. For the Webpage dataset, the balanced accuracy and Gmean increase by 0.8% and 0.6% than the SMOTE oversampling method. However, the MCC and F1-Score metric values improved by 10.3% and 0.5% than Kmean SMOTE.

We have also compared RCS against ISMOTE [4] accross all the tabular datasets. On the Contraceptive dataset, **RCS** clearly outperforms **ISMOTE** across all metrics, achieving higher BACC (0.555 vs. 0.476), MCC (0.329 vs. 0.205), and F1-Score (0.546 vs. 0.430). The consistent gains and lower variance demonstrate superior robustness and improved handling of class overlap in highly imbalanced settings. For the high-dimensional Isolet dataset, **RCS** consistently surpasses **ISMOTE**, achieving near-optimal performance with BACC of 0.997 and MCC of 0.976. While ISMOTE attains reasonable accuracy, RCS exhibits stronger class separability and lower variance, indicating better preservation of inter-class structure in large-scale multi-class scenarios. On the Webpage dataset, **RCS** significantly improves upon **ISMOTE**, particularly in MCC (0.879 vs. 0.682), while also achieving higher BACC (0.938 vs. 0.914). These results reflect more accurate decision boundary modeling and improved minority-class discrimination in datasets with complex feature dependencies. In the Optical_digits dataset, **RCS** consistently outperforms **ISMOTE**, achieving higher BACC (0.991 vs. 0.933) and MCC (0.942 vs. 0.743). This indicates superior discriminative representation learning and reduced sensitivity to noise compared to synthetic oversampling. For the Dermatology dataset, **ISMOTE** exhibits unstable behavior, particularly in G-Mean, whereas **RCS** achieves uniformly strong performance across all metrics, including BACC (0.981), MCC (0.979), and F1-Score (0.983). This highlights the robustness of RCS in handling heterogeneous multi-class imbalance. On the Wine dataset, **RCS** shows a pronounced advantage over **ISMOTE**, delivering substantial improvements in BACC (0.950 vs. 0.770) and MCC (0.920 vs. 0.670) with significantly lower variance.

See Supplementary Section S4 for **Statistical tests**.

5. Ablation study

We perform several ablation experiments to conduct a thorough analysis of our calibration method.

5.1. Visualization of the proposed RCS framework through t-SNE plots

Table 9 presents the t-distributed Stochastic Neighbor Embedding (t-SNE) plots of latent vectors from MNIST, FMNIST, CIFAR10, and SVHN. Both the DeepSMOTE Autoencoder and our Autoencoder were trained for 200 epochs. After training, the imbalanced datasets were encoded into latent feature vectors, oversampled to mitigate imbalance, and then projected into two dimensions via t-SNE, with distinct colors assigned to each class.

The plots clearly show that our Autoencoder achieves better class separation than the DeepSMOTE Autoencoder under imbalanced conditions. Post-oversampling, our calibration method produced samples that aligned closely with their respective classes while preserving inter-class separation.

For simpler datasets (MNIST, FMNIST), classes remained linearly separable both before and after oversampling, reflecting the robustness of the proposed RCS framework. For more complex datasets (SVHN, CIFAR10), classes were well separated prior to oversampling, though some overlap appeared afterward. Even so, our oversampling approach outperformed state-of-the-art methods in accuracy, underscoring its effectiveness. Additionally, in Supplementary Section S5.10, presents the **Clustering Separability Index**, using the Davies-Bouldin Index and Silhouette Score to empirically assess the disentanglement of latent feature vectors before and after oversampling.

5.2. Visualization of the generated images obtained by cDCGAN, DeepSMOTE, and RCS oversampling method

The images from various oversampling methods were illustrated in Table 10. Our RCS method posed a different strategy to generate synthetic samples than the other oversampling methods. Moreover, cDCGAN performed poorly over these datasets. It is observed that these methods perform better in MNIST and FMNIST than SVHN and CIFAR10, as the former has a single channel and is easily represented in the feature space. Even though complex datasets such as CIFAR10 and SVHN did not generate images as accurate as ground-truth images, a better accuracy than state-of-the-art methods has been observed, as we have discussed earlier. We visualized the generated images for MNIST, FMNIST, SVHN, and CIFAR10 datasets for Original Image, Conditional DCGAN, DeepSMOTE, and RCS Method. We did not include the TailCalibX calibration method since the generated images were not good enough.

To further validate the robustness of the proposed RCS framework, we conducted extensive ablation studies analyzing the impact of key hyperparameters, architectural components, and design choices; **detailed ablation study** is provided in the supplementary material section S5.

6. Conclusion and future work

Class imbalance remains a fundamental challenge even when classes are structurally distinguishable, since limited minority samples often lead to poor estimation of class distributions and unstable decision boundaries. Under such imbalanced conditions, conventional training paradigms tend to favor the majority class, thereby impeding the effective learning of discriminative representations for minority class samples. To address this, an end-to-end RCS framework specifically designed for imbalanced learning scenarios has been proposed, which integrates disentangled representation learning with statistically calibrated subclass-level sampling. At its core, we train an autoencoder to preserve class separability by ensuring a smooth transition of structure from the data space to the latent space. To enhance prior efforts in distribution calibration, we introduce a novel calibration strategy that emphasizes the local statistical properties of feature vectors corresponding to minority class instances, while also incorporating intermediate class samples through targeted oversampling. The proposed RCS algorithm consistently outperforms existing SOTA methods across multiple datasets and modalities. Beyond empirical gains, this work shows distribution calibration's potential for latent-space synthetic sample generation. The proposed RCS framework assumes approximate Gaussianity of latent subclasses, an assumption that may be partially violated in scenarios involving highly complex or strongly non-Gaussian data distributions. Although shrinkage-based covariance regularization ensures numerical stability and robust estimation under data scarcity, the selection of Gaussian mixture components introduces additional hyperparameters that may influence performance and require careful tuning in certain applications. Furthermore, the proposed approach incurs a higher computational cost compared to simple reweighting strategies; however, it remains substantially more computationally efficient than adversarial or generative oversampling methods while providing improved distributional consistency and robustness. These considerations highlight several promising directions for future research aimed at further enhancing the scalability, adaptability, and modeling flexibility of the proposed RCS framework. In particular, future work will focus on extending RCS to federated and continual imbalance learning settings,

where data distributions evolve over time or remain decentralized across multiple sources. Another important direction involves deriving theoretical generalization bounds for calibrated subclass models, thereby providing a deeper analytical understanding of their performance guarantees. Additionally, incorporating robustness to label noise represents a meaningful extension, especially for real-world datasets where annotation uncertainty is unavoidable. Finally, developing uncertainty-aware calibration strategies for extreme long-tail distributions may further improve reliability and decision confidence in highly imbalanced scenarios.

CRediT authorship contribution statement

Priyobrata Mondal: Writing – original draft, Software, Methodology, Investigation, Formal analysis; **Faizanuddin Ansari:** Writing – original draft, Resources, Methodology, Investigation, Formal analysis, Conceptualization; **Swagatam Das:** Writing – review & editing, Visualization, Validation, Supervision, Methodology, Conceptualization.

Data availability

Public Datasets were used.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at 10.1016/j.inffus.2026.104331

References

[1] K.A. Shah, Z. Halim, S. Anwar, C.-C. Hsu, I. Rida, Multi-sensor data fusion for smart healthcare: optimizing specialty-based classification of imbalanced EMRs, Inf. Fusion 125 (2026) 103503.

[2] M.S. Santos, P.H. Abreu, N. Japkowicz, A. Fernández, J. Santos, A unifying view of class overlap and imbalance: key concepts, multi-view panorama, and open avenues for research, Inf. Fusion 89 (2023) 228–253.

[3] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: Synthetic minority over-sampling technique, J. Artif. Intell. Res. 16 (2002) 321–357.

[4] Y. Li, Y. Yang, P. Song, L. Duan, R. Ren, An improved SMOTE algorithm for enhanced imbalanced data classification by expanding sample generation space, Sci. Rep. 15 (1) (2025) 23521.

[5] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, Adv. Neural Inf. Process. Syst. 27 (2014).

[6] D.P. Kingma, M. Welling, Auto-encoding variational bayes, arXiv preprint arXiv:arXiv:1312.6114 (2013).

[7] S.S. Mullick, S. Datta, S. Das, Generative adversarial minority oversampling, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 1695–1704.

[8] X. Wang, Y. Lyu, L. Jing, Deep generative model for robust imbalance classification, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 14124–14133.

[9] D. Dablain, B. Krawczyk, N.V. Chawla, DeepSMOTE: fusing deep learning and SMOTE for imbalanced data, IEEE Trans. Neural Netw. Learn. Syst. 34 (9) (2023) 6390–6404.

[10] X. Wang, L. Jing, Y. Lyu, M. Guo, J. Wang, H. Liu, J. Yu, T. Zeng, Deep generative mixture model for robust imbalance classification, IEEE Trans. Pattern Anal. Mach. Intell. 45 (3) (2022) 2897–2912.

[11] Y. Li, J. Jin, H. Tao, Y. Xiao, J. Liang, C.L.P. Chen, Complemented subspace-based weighted collaborative representation model for imbalanced learning, Appl. Soft Comput. 153 (2024) 111319.

[12] A.R. Chowdhury, A. Gupta, S. Das, Deep multi-view clustering: a comprehensive survey of the contemporary techniques, Inf. Fusion 119 (2025) 103012.

[13] J. Jin, S. Chang, J. Duan, Y. Li, W. Ding, Z. Wang, C.L.P. Chen, P. Li, Groupwise label enhancement broad learning system for image classification, IEEE Trans. Cybern. (2025).

[14] Y. Li, Y. Wang, J. Jin, W. Zhang, H. Tao, H. Wu, C.L.P. Chen, Imbalanced broad learning system with label relaxation and sample weight adaptation, Appl. Soft Comput. 182 (2025) 113543.

[15] S. Das, S.S. Mullick, I. Zelinka, On supervised class-imbalanced learning: an updated perspective and some key challenges, IEEE Trans. Artif. Intell. 3 (6) (2022) 973–993.

[16] H. Zhang, M. Li, RWO-Sampling: A random walk over-sampling approach to imbalanced data classification, Inf. Fusion 20 (2014) 99–116.

[17] H. Han, W.-Y. Wang, B.-H. Mao, Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning, in: International Conference on Intelligent Computing, Springer, 2005, pp. 878–887.

[18] H. He, Y. Bai, E.A. Garcia, S. Li, ADASYN: Adaptive synthetic sampling approach for imbalanced learning, in: 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence), 2008, pp. 1322–1328.

[19] M. Mirza, S. Osindero, Conditional Generative Adversarial Nets, 2014. 1411.1784

[20] G. Mariani, F. Scheidegger, R. Istrate, C. Bekas, C. Malossi, Bagan: data augmentation with balancing gan, arXiv preprint arXiv:arXiv:1803.09655 (2018).

[21] A.K. Mondal, L. Singhal, P. Tiwary, P. Singla, P. AP, Minority oversampling for imbalanced data via class-preserving regularized auto-encoders, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2023, pp. 3440–3465.

[22] S. Yang, L. Liu, M. Xu, Free lunch for few-shot learning: distribution calibration, in: International Conference on Learning Representations, 2021.

[23] R. Vigneswaran, M.T. Law, V.N. Balasubramanian, M. Tapaswi, Feature generation for long-tail classification, in: Proceedings of the Twelfth Indian Conference on Computer Vision, Graphics and Image Processing, 2021, pp. 1–9.

[24] F. Ansari, S. Das, P. Shamsolmoali, Handling class imbalance by estimating minority class statistics, in: 2023 International Joint Conference on Neural Networks (IJCNN), 2023, pp. 1–8.

[25] R. Salakhutdinov, J. Tenenbaum, A. Torralba, One-shot learning with a hierarchical nonparametric bayesian model, in: Proceedings of ICML Workshop on Unsupervised and Transfer Learning, JMLR Workshop and Conference Proceedings, 2012, pp. 195–206.

[26] I. Higgins, D. Amos, D. Pfau, S. Racaniere, L. Matthey, D. Rezende, A. Lerchner, Towards a definition of disentangled representations, arXiv preprint arXiv:arXiv:1812.02230 (2018).

[27] J. Cha, J. Thiyagalingam, Disentangling autoencoders (DAE), arXiv preprint arXiv:2202.09926 (2022).

[28] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, D. Krishnan, Supervised contrastive learning, Adv. Neural Inf. Process. Syst. 33 (2020) 18661–18673.

[29] J. Jin, B. Geng, Y. Li, J. Liang, Y. Xiao, C.L.P. Chen, Flexible label-induced manifold broad learning system for multiclass recognition, IEEE Trans. Neural Netw. Learn. Syst. (2023).

[30] J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel, The german traffic sign recognition benchmark: a multi-class classification competition, in: The 2011 International Joint Conference on Neural Networks, IEEE, 2011, pp. 1453–1460.

[31] A. Coates, A. Ng, H. Lee, An analysis of single-layer networks in unsupervised feature learning, in: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, JMLR Workshop and Conference Proceedings, 2011, pp. 215–223.

[32] M.A.H. Farquad, I. Bose, Preprocessing unbalanced data using support vector machine, Decis. Support Syst. 53 (1) (2012) 226–233.

[33] D.A. Cieslak, N.V. Chawla, A. Striegel, Combating imbalance in network intrusion datasets, in: 2006 IEEE International Conference on Granular Computing, 2006, pp. 732–737.

[34] L. Ma, S. Fan, CURE-SMOTE Algorithm and hybrid algorithm for feature selection and parameter optimization based on random forests, BMC Bioinform. 18 (1) (2017) 1–18.

[35] C. Bunkhumpornpat, K. Sinapiromsaran, C. Lursinsap, DBSMOTE: Density-based synthetic minority over-sampling technique, Appl. Intell. 36 (3) (2012) 664–684.

[36] Z. Xu, D. Shen, T. Nie, Y. Kou, N. Yin, X. Han, A cluster-based oversampling algorithm combining SMOTE and k-means for imbalanced medical data, Inf. Sci. 572 (2021) 574–589.

[37] A.S. Ghorab, W.M. Ashour, S.I. Abudalfa, An adaptive oversampling method for imbalanced datasets based on mean-shift and SMOTE, in: Explore Business, Technology Opportunities and Challenges after the Covid-19 Pandemic, Springer, 2022, pp. 13–23.

[38] S. Barua, M.M. Islam, X. Yao, K. Murase, MWMOTE–Majority Weighted minority oversampling technique for imbalanced data set learning, IEEE Trans. Knowl. Data Eng. 26 (2) (2014) 405–425.

[39] A. Sánchez, E. Morales, J. Gonzalez, Synthetic oversampling of instances using clustering, Int. J. Artif. Intell. Tools 22 (2013).

[40] S. Chen, G. Guo, L. Chen, A new over-sampling method based on cluster ensembles, in: 2010 IEEE 24Th International Conference on Advanced Information Networking and Applications Workshops, IEEE, 2010, pp. 599–604.